Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

Introdução à informática

Endereçamento e protocolos de rede

Aula 1: Endereçamento IP: IPv4 e IPv6

Código da aula: DADOSANO1C1B2S8A1

GOVERNO DO ESTADO
DE SÃO PAULO

Introdução à informática

Mapa da Unidade 1 Componente 1

Introdução à informática

Mapa da Unidade 1 Componente 1

Você está aqui!

Endereçamento e protocolos de rede

8

Aula 1: Endereçamento IP: IPv4 e IPv6

Código da aula: DADOSANO1C1B2S8A1

Objetivos da aula

- Compreender e aplicar o conceito de endereçamento IP: IPv4 e IPv6.

Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e/ou à internet.
- Papel e caneta.

Duração da aula

50 minutos.

Habilidades técnicas

- Interpretar endereçamento IP e funcionamento de protocolos de rede.

Habilidades socioemocionais

- Interagir com confiança e disposição para aprender com os outros.

Ponto de **partida**

Endereço único

Imagine uma situação em que você e seus amigos estão enviando mensagens pelo celular usando o WhatsApp.

Já parou para pensar em como o aplicativo consegue descobrir exatamente para qual celular deve enviar cada mensagem, mesmo havendo milhões de usuários conectados no mundo todo?

© Getty Images

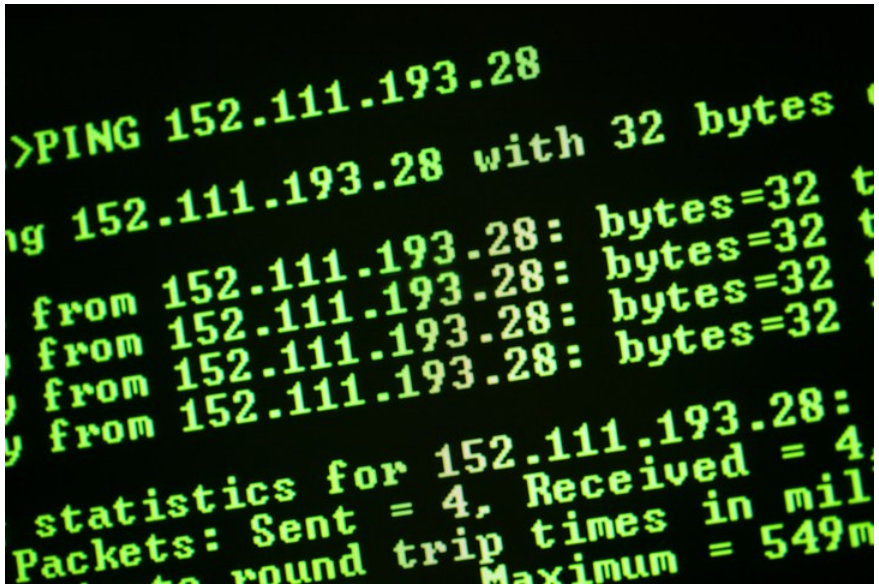
- Como você acha que os computadores e celulares “sabem” para onde mandar as mensagens dentro da rede?

Situação fictícia produzida pela SEDUC-SP.

Construindo o **conceito**

Endereço IP: a identidade digital

O **IP (Internet Protocol)** funciona como o endereço de uma casa, mas no mundo digital.
Ele permite que computadores, celulares e servidores se localizem e troquem informações na rede. Por isso, cada dispositivo conectado precisa de um **endereço único** para se comunicar.



Fonte: MONTEIRO, 2019.

© Getty Images

Curiosidade

Sem IP, seria impossível saber para onde enviar dados na internet.

Construindo o **conceito**

IPv4 e IPv6: duas versões do IP

IPv4 (Internet Protocol version 4)

- 32 bits
- formato decimal com 4 blocos.
- **Ex.:** 192.168.0.1
- suporta cerca de **4 bilhões de endereços**, mas já não é suficiente.

IPv6 (Internet Protocol version 6)

- 128 bits
- formato hexadecimal com 8 blocos.
- **Ex.:** 2001:0db8:85a3::8a2e:0370:7334
- foi criado para **garantir endereços ilimitados** e atender à expansão da internet e da IoT (Internet das Coisas).

DESTAQUE

O IPv6 é a evolução natural do IPv4, garantindo espaço para o futuro digital.

Construindo o **conceito**

Máscara de sub-rede: como separar grupos

A **máscara de sub-rede** funciona como uma “linha divisória” dentro do IP. Ela organiza os dispositivos, mostrando quem está no mesmo grupo:

- qual parte se refere à **rede** (grupo de aparelhos juntos);
- qual parte se refere ao **dispositivo** (cada aparelho dentro desse grupo).

Todos os aparelhos que começam com 192.168.0. estão no **mesmo grupo**.
O número final (.15) é o “RG” do aparelho dentro da rede.

IP do PC	192.168.0.15
Máscara	255.255.255.0

Com essa máscara, qualquer IP que comece com 192.168.0.x (em que 'x' é um número diferente de 0 e 255) estará na mesma rede 192.168.0.0.

Produzido pela SEDUC-SP

Construindo o **conceito**

Exemplo de máscara de sub-rede

Se você tem **192.168.0.10/24** (máscara **255.255.255.0**) e seu colega tem **192.168.0.25/24**, vocês estão na **mesma sub-rede (192.168.0.0/24)** e conseguem trocar arquivos diretamente.

Se o seu colega estiver com **192.168.1.20/24**, a máscara **255.255.255.0** indica que ele está na **sub-rede 192.168.1.0/24**, diferente da sua (**192.168.0.0/24**).

Nesse caso, vocês não estão no mesmo grupo de rede e precisam de um roteador para se comunicar.

DESTAQUE

A máscara ajuda a saber quem pode conversar direto dentro da rede.

Construindo o **conceito**

Classes de IP: tamanhos diferentes de grupos

Classe	Descrição	Exemplo
A	Redes muito grandes podem ter milhões de dispositivos.	Uma rede de uma grande empresa ou até de um país.
B	Redes médias podem ter milhares de dispositivos.	Rede de uma escola ou de uma universidade.
C	Redes pequenas, até algumas centenas de dispositivos.	Rede da sua casa ou do Wi-Fi da sala de aula.

Construindo o **conceito**

Classes especiais e CIDR

Classe/Sistema	Explicação	Exemplo
D	Usada para transmissões em grupo (*multicast*).	Uma live que envia o mesmo vídeo para várias pessoas ao mesmo tempo.
E	Reservada para pesquisas e testes.	Cientistas e técnicos de redes.
CIDR (Classless Inter-Domain Routing – Roteamento sem Classe)	Sistema atual que substituiu as classes, mais flexível.	Em roteadores de casa aparece como /24, indicando uma rede pequena (até 254 dispositivos).

Registro

Pause e **responda**

Qual é a principal função de um endereço IP (Internet Protocol) em uma rede de computadores?

- Distribuir a conexão.
- Guardar senhas da rede.
- Controlar a velocidade.
- Identificar o dispositivo.

Registro

Pause e **responda**

Qual é a principal função de um endereço IP (Internet Protocol) em uma rede de computadores?

- ✗ Distribuir a conexão.
- ✗ Guardar senhas da rede.
- ✗ Controlar a velocidade.
- ✓ Identificar o dispositivo.

Colocando em **prática**

Verificando a rede da escola

Na sala de informática da escola, alguns computadores não conseguem acessar a internet. Você foi chamado para ajudar a identificar o problema. Os técnicos passaram os seguintes endereços configurados:

- PC1 = 192.168.10.15
- PC2 = 192.168.10.200
- PC3 = 192.168.11.5
- A rede da escola usa a máscara **255.255.255.0 (/24)**.

Situação fictícia produzida pela SEDUC-SP.

18 minutos

Em duplas

Tarefa

Em duplas, respondam em uma folha:

1. Esses três computadores podem se comunicar entre si?
2. Se não, quais estão juntos e qual ficou de fora?
3. Qual é a classe dessa rede (A, B ou C)?

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. O endereço IP (Internet Protocol) é como o “endereço digital” de cada dispositivo na rede, essencial para que dados cheguem ao destino correto.

2. Existem duas versões: IPv4, mais antigo e limitado, e IPv6, que garante praticamente infinitos endereços para o futuro da internet.

3. As máscaras de sub-rede e as classes de IP ajudam a organizar as redes em grupos de tamanhos diferentes, tornando a comunicação mais eficiente.

Saiba mais

Quer entender como seu Wi-Fi conecta os colegas de jogo? Desvende o segredo das máscaras de rede e domine a conexão, assistindo a esse vídeo imperdível.

DICIONARIOTEC. O que são a máscara de rede e sub-rede e como afetam o endereço IP!? YouTube, 17 jul. 2023. Disponível em: https://www.youtube.com/watch?v=Tox1UY29sJU. Acesso em: 14 out. 2025.

Referências da aula

BROOKSHEAR, J. G. **Ciência da computação**: uma visão abrangente. Porto Alegre: Bookman, 2013.

MONTEIRO, M. A. **Introdução à organização de computadores**. Rio de Janeiro: LTC, 2019.

STALLINGS, W. **Arquitetura e organização de computadores**. São Paulo: Pearson Education do Brasil, 2017.

TANENBAUM, A. S.; AUSTIN, T. **Organização estruturada de computadores**. São Paulo: Pearson Prentice Hall, 2013.

Identidade visual: imagens © Getty Images

Orientações ao professor

Slide 5

Tempo previsto:
Ponto de partida: 8 minutos
Construindo o conceito: 20 minutos
Pause e responda: 2 minutos
Colocando em prática: 18 minutos
O que nós aprendemos hoje?: 1 minuto
Saiba mais: 1 minuto

Slide 6

Orientações: professor, a seção **Ponto de partida** tem como objetivos ativar o conhecimento prévio dos estudantes sobre o tema da aula e estimular seu pensamento crítico e suas habilidades comunicativas. Por meio de uma situação-problema ou exemplo próximo da realidade dos estudantes, pretende-se sair da abstração conceitual e promover um diálogo dinâmico para explorar hipóteses, soluções e compartilhar eventuais experiências que eles já possam ter com os tópicos a serem abordados na aula. Também é um momento de engajá-los em relação ao tema da aula.

Tempo previsto: 8 minutos.

Gestão de sala de aula: assegure que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos, para garantir a participação de todos. Mantenha um ambiente de respeito, no qual todas as opiniões sejam valorizadas, garantindo que todos se sintam confortáveis para expressar seus pontos de vista. Conclua a atividade resumindo as principais ideias discutidas e vinculando-as aos objetivos de aprendizagem da aula.

Condução da dinâmica: apresente o contexto para os alunos:
Discussão da pergunta (5 minutos): apresente a questão e dê tempo para os alunos pensarem e discutirem.
Apresentação (3 minutos): peça para alguns alunos compartilharem suas ideias com a turma.

Expectativas de respostas: não há resposta certa ou errada. Os alunos podem sugerir explicações como:
“O celular deve ter um número de identificação.”
“O aplicativo escolhe o caminho automaticamente.”
“Existe um código que indica o aparelho correto.”
O importante é que percebam que deve haver um **endereço único** que identifica cada dispositivo conectado à rede.

Slides 7 a 12

Tempo previsto: 20 minutos.

Gestão de sala de aula: professor, inicie a seção **Construindo o conceito** criando um ambiente relaxado e convidativo para um diálogo aberto. Encoraje a participação de todos os estudantes, garantindo que cada voz possa ser ouvida. Caso surjam respostas longas ou debates paralelos, delicadamente redirecione a conversa para o tópico original.

Condução da dinâmica: durante a explicação, lembre-se de que os alunos estão tendo o **primeiro contato com o tema**. O objetivo é que eles saiam da aula entendendo **o que é um endereço IP, por que ele existe, quais são as versões principais (IPv4 e IPv6) e como funcionam as máscaras e classes**.

Comece com a ideia de **endereço físico**, como o da casa ou da escola, para que os estudantes percebam que, assim como cada pessoa precisa de um endereço para receber cartas ou encomendas, cada **dispositivo na internet precisa de um endereço IP para enviar e receber informações**. Isso cria uma ponte entre a vida cotidiana e o conteúdo técnico.

Explique a diferença entre **IPv4 (Internet Protocol version 4)** e **IPv6 (Internet Protocol version 6)** usando exemplos visuais: mostre um endereço simples de IPv4 (ex.: 192.168.0.1) e compare com o de IPv6 (ex.: 2001:0db8:85a3::8a2e:0370:7334). Ressalte que o IPv4 já está ficando cheio, e o IPv6 veio para garantir que sempre haja novos endereços disponíveis.

Na sequência, introduza de forma simples o conceito de **máscara de sub-rede**, mostrando que ela serve para separar o que é a **rede (grupo)** e o que é o ***host* (o aparelho)**. Use o exemplo do **Wi-Fi da escola**: todos os alunos estão na mesma rede, mas cada celular tem o seu número de identificação.

Apresente as **classes de IP** de maneira comparativa (A, B e C), destacando que elas indicavam se uma rede era muito grande, média ou pequena. Mostre exemplos próximos: **Classe A** poderia ser usada por uma grande empresa, **Classe B** por uma universidade, e **Classe C** pelo Wi-Fi da escola ou de casa. Inclua também as classes D e E apenas como curiosidade e diga que hoje usamos mais o **CIDR (Classless Inter-Domain Routing)**, que aparece em roteadores como "/24".

Conclua mostrando aos alunos **como encontrar o próprio IP no celular ou no computador** e explique que esse conhecimento é útil não só para resolver problemas de conexão, mas também para quem quer ser **cientista de dados**, já que endereços IP aparecem em registros (*logs*), análises de tráfego de rede e no tratamento de grandes volumes de dados.

Slides 13 e 14

Orientações: responder ao quiz. Há apenas uma alternativa correta.

Tempo previsto: 2 minutos.

Pergunta: Qual é a principal função de um endereço IP (Internet Protocol) em uma rede de computadores?
Resposta correta: Identificar o dispositivo.
Feedback: o IP identifica cada dispositivo na rede, garantindo que os dados cheguem ao destino correto. As outras funções (velocidade, senhas e distribuição) pertencem a outros recursos da rede.

Slide 15

Orientações: professor, a seção **Colocando em prática** tem como objetivo aplicar os conhecimentos construídos durante a aula, incentivando os estudantes a pensar criticamente e de forma prática.

Tempo previsto: 18 minutos.

Gestão de sala de aula: Introdução (2 minutos): apresente o contexto como uma situação real de suporte em redes escolares: às vezes, um computador não consegue acessar a rede porque está configurado com IP errado.

Condução da dinâmica:

Atividade em duplas (12 minutos):
Os alunos devem analisar os três endereços e decidir quem está no mesmo grupo de rede.
Depois, justificar a resposta e indicar a classe correspondente. Os alunos devem anotar as respostas em uma folha.
Circule pela sala observando o raciocínio e auxiliando quando necessário.
Sorteio e apresentação das respostas (4 minutos):
Escolha duas ou três duplas para compartilharem suas respostas oralmente.

Expectativas de respostas:
1. **PC1 e PC2** estão no mesmo grupo de rede (**192.168.10.0/24**).
2. **PC3** está fora, porque pertence a outra rede (**192.168.11.0/24**).
3. Essa rede é da **Classe C**, usada em ambientes pequenos, como escolas e residências.

Slide 16

Orientações: professor, a seção **O que nós aprendemos hoje?** tem como objetivos reforçar e esclarecer os conceitos principais discutidos na aula. Essa revisão pode ser uma ferramenta de avaliação informal do aprendizado dos estudantes, identificando áreas que possam precisar de mais atenção em aulas futuras.

Tempo previsto: 1 minuto.

Gestão de sala de aula:
Mantenha um tom positivo e construtivo, reforçando o aprendizado em vez de focar correções.
Seja direto e objetivo nas explicações, para manter a atividade dentro do tempo estipulado.
Engaje os estudantes rapidamente, pedindo confirmações ou reações breves às definições apresentadas.

Condução da dinâmica:
Explique que esta parte da seção, “Então ficamos assim...”, é um momento de reflexão e de esclarecimento sobre os conceitos abordados na aula.
Informe que será uma rápida revisão para assegurar que os entendimentos dos estudantes estejam alinhados com as definições corretas dos conceitos.
Apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando-o em forma de frases completas.
Destaque se as contribuições dos estudantes estavam alinhadas com os conceitos e ofereça esclarecimentos rápidos caso haja discrepâncias ou mal-entendidos.
Finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.
Reforce a ideia de que essa revisão ajuda a solidificar o entendimento dos estudantes e a prepará-los para aplicar esses conceitos em situações práticas.

Expectativas da atividade:
Os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.
A atividade serve como uma verificação rápida do entendimento dos estudantes e uma oportunidade para corrigir quaisquer mal-entendidos.

Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

Introdução à informática

Endereçamento e protocolos de rede

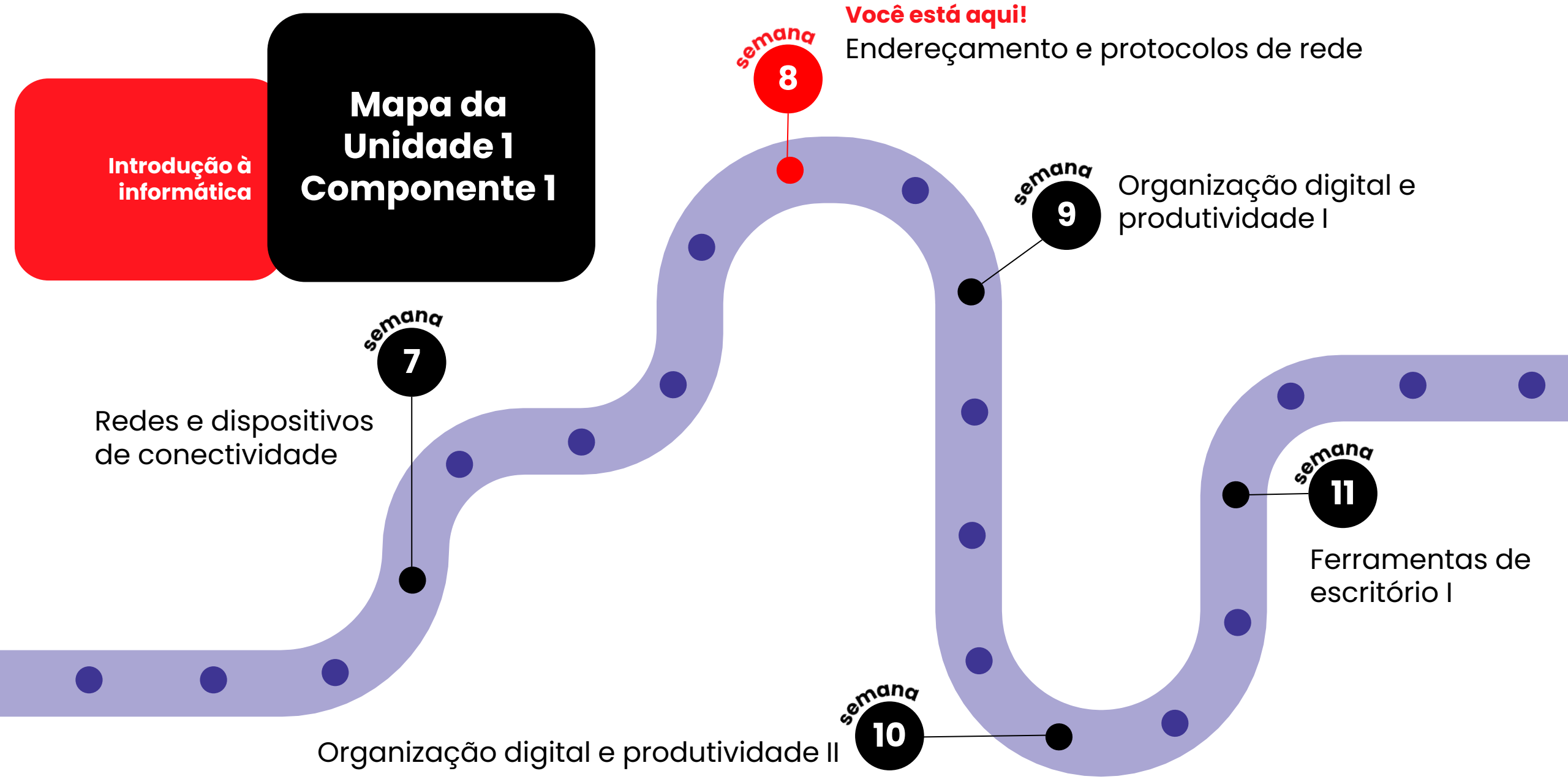
Aula 2: Protocolos: TCP/IP, UDP, DNS

Código da aula: DADOSANO1C1B2S8A2

GOVERNO DO ESTADO
DE SÃO PAULO

Introdução à informática

Mapa da Unidade 1 Componente 1



Introdução à informática

Mapa da Unidade 1 Componente 1

Você está aqui!

Endereçamento e protocolos de rede

8

Aula 2: Protocolos: TCP/IP, UDP, DNS

Código da aula: DADOSANO1C1B2S8A2

Objetivos da aula

- Compreender e aplicar o conceito de protocolos: TCP/IP, UDP, DNS.

Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e/ou à internet.
- Papel e caneta.

Duração da aula

50 minutos.

Habilidades técnicas

- Interpretar endereçamento IP e funcionamento de protocolos de rede.

Habilidades socioemocionais

- Interagir com confiança e disposição para aprender com os outros.

Relembre

Endereço IP

- Para que serve o endereço IP em uma rede?
- Qual é a diferença entre IPv4 e IPv6?
- O que indica a máscara de sub-rede?

Construindo o **conceito**

Protocolos de rede: as regras da comunicação

- é o conjunto de regras que permite a troca de dados entre computadores;
- definem **como os dados são enviados, recebidos e interpretados**;
- são a **base da internet**, pois garantem ordem e organização no tráfego de dados.

DESTAQUE

Protocolos são as regras que tornam a internet possível. Sem protocolos, cada máquina teria sua própria "língua" e não se comunicaria.

Fonte: STALLINGS, 2017.

Construindo o **conceito**

TCP/IP: a base da internet

TCP/IP (Transmission Control Protocol/Internet Protocol) é o conjunto de protocolos que estrutura a internet. Ele organiza a comunicação em **camadas** (aplicação, transporte, rede e enlace). Cada camada tem funções específicas, pois ao **dividir tarefas, evita confusões**.

DESTAQUE

Permite que diferentes tecnologias (Wi-Fi, cabo, fibra, 4G) funcionem juntas.

Exemplo:

Ao mandar uma foto no WhatsApp:

- ✓ uma camada **(aplicação)** cuida do envio;
- ✓ outra camada **(transporte)** garante que os dados cheguem completos;
- ✓ outras camadas **(rede e enlace)** definem a rota e entregam a mensagem. Essas camadas também dividem a foto em pacotes menores, que são enviados pela rede e depois reorganizados no destino.

Fonte: TANENBAUM; AUSTIN, 2013.

Construindo o **conceito**

TCP: entrega confiável de dados

O **TCP** estabelece uma **conexão** antes de transmitir os dados. A entrega ocorre em três passos:

1

Conexão criada

Antes de enviar qualquer dado, o TCP estabelece uma “conversa inicial” entre os dois computadores.

Isso garante que ambos estejam prontos para trocar informações.

2

Pacotes enviados e verificados

A informação é dividida em pequenos pacotes.

O TCP numera cada pacote e confirma se todos chegaram ao destino na ordem correta.

3

Reenvio quando necessário

Se algum pacote se perder ou chegar com erro, o TCP pede o reenvio automaticamente.

Assim, o destinatário recebe a informação completa, sem falhas.

Fonte: TANENBAUM; AUSTIN, 2013.

Construindo o **conceito**

UDP: envio rápido de dados

UDP (User Datagram Protocol) é o protocolo da “velocidade”. É usado quando a fluidez no envio dos dados importa mais do que a confiabilidade.

O envio pelo UDP é mais rápido, mas pode haver perdas (aceitáveis em contextos de tempo real). Seu foco é no **menor atraso**. Na tabela a seguir, é possível comparar o envio de dados **com TCP** e **com UDP**:

Com TCP	Com UDP
Confere se tudo chegou	Não verifica pacotes
Mais seguro	Mais rápido
Usado em downloads de arquivos	Usado em jogos on-line e transmissões ao vivo

Fonte: TANENBAUM; AUSTIN, 2013.

Construindo o **conceito**

DNS: traduzindo nomes em números

O **DNS (Domain Name System)** é um sistema que:

- transforma nomes em endereços IP; sem ele, precisaríamos decorar números como 142.250.190.78 em vez de google.com;
- funciona como uma **agenda de contatos**: você procura o nome, ele devolve o número;
- é distribuído em servidores no mundo inteiro, para que sempre haja resposta.

Fonte: TANENBAUM; AUSTIN, 2013.

Tome nota

O DNS facilita a navegação porque transforma nomes de sites em números que os computadores entendem.

Construindo o **conceito**

Como tudo se conecta

	Função na rede	Exemplo
Protocolo	Define as regras de comunicação entre dispositivos, como uma “língua em comum”.	Duas pessoas que falam a mesma língua conseguem conversar sem dificuldades.
TCP/IP	Estrutura de protocolos que organiza a internet em camadas, garantindo que diferentes tecnologias funcionem juntas.	Você pode assistir a uma série no celular (Wi-Fi), no notebook (cabo) ou na TV (fibra), e tudo se integra sem problemas.
TCP	Foca a confiabilidade: garante que os dados cheguem completos e na ordem certa.	Ao enviar um currículo por e-mail, você tem certeza de que o documento chega inteiro ao recrutador.
UDP	Foca a velocidade: envia os dados sem checar se todos chegaram.	Em uma chamada de vídeo, pode haver pequenas falhas na imagem, mas a conversa flui sem travar.
DNS	Traduz nomes em endereços IP, facilitando a navegação.	Em vez de digitar números longos, você pesquisa “spotify.com” e acessa a página rapidamente.

Registro

Pause e **responda**

Qual protocolo é usado quando a prioridade é transmitir dados rapidamente, mesmo que alguns pacotes se percam no caminho?

- DNS
- UDP
- IP
- TCP

Registro

Pause e **responda**

Qual protocolo é usado quando a prioridade é transmitir dados rapidamente, mesmo que alguns pacotes se percam no caminho?

- ✗ DNS
- ✓ UDP
- ✗ IP
- ✗ TCP

Colocando em **prática**

Registro

Escolhendo o protocolo certo

Você faz parte da equipe de TI de uma **empresa de tecnologia** que está implantando novos serviços de rede. Sua tarefa é auxiliar o time a identificar **qual protocolo deve ser utilizado em cada situação**:

1. Reunião on-line semanal com todos os colaboradores, transmitida em vídeo.
2. Envio de relatórios financeiros por e-mail para os diretores.
3. Acesso ao portal da empresa, digitando o nome no navegador.

Situação fictícia produzida pela SEDUC-SP.

Tarefa

Em duplas, discutam e anotem em uma folha:

- Qual dispositivo de rede é o mais adequado para cada situação?
- Justifiquem sua escolha, explicando sua função.

Na aula

Em duplas

Documento

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Os protocolos de rede são regras que organizam a comunicação entre computadores e tornam a internet possível.

2. O TCP/IP estrutura a troca de dados, enquanto o TCP foca na confiabilidade e o UDP na velocidade.

3. O DNS facilita o uso da internet ao traduzir nomes de sites em endereços numéricos compreendidos pelos computadores.

Saiba mais

Você já se perguntou como seus dados viajam na internet sem se perder? Descubra os guardiões da informação, TCP e UDP, e domine os segredos da camada de transporte, assistindo a esse vídeo incrível.

DICIONÁRIO DE INFORMÁTICA. TCP UDP e os protocolos da camada de transporte. YouTube, 4 ago. 2021. Disponível em: https://www.youtube.com/watch?v=Wo7GOmuN-og. Acesso em: 14 out. 2025.

Referências da aula

BROOKSHEAR, J. G. **Ciência da computação**: uma visão abrangente. Porto Alegre: Bookman, 2013.

MONTEIRO, M. A. **Introdução à organização de computadores**. Rio de Janeiro: LTC, 2019.

STALLINGS, W. **Arquitetura e organização de computadores**. São Paulo: Pearson Education do Brasil, 2017.

TANENBAUM, A. S.; AUSTIN, T. **Organização estruturada de computadores**. São Paulo: Pearson Prentice Hall, 2013.

Identidade visual: imagens © Getty Images

Orientações ao professor

Slide 5

Tempo previsto:
Relembre: 8 minutos
Construindo o conceito: 20 minutos
Pause e responda: 2 minutos
Colocando em prática: 18 minutos
O que nós aprendemos hoje?: 1 minuto
Saiba mais: 1 minuto

Slide 6

Tempo previsto: 8 minutos.

Gestão de sala de aula: assegure que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos, para garantir a participação de todos. Mantenha um ambiente de respeito, no qual todas as opiniões sejam valorizadas, garantindo que todos se sintam confortáveis para expressar seus pontos de vista.

Condução da dinâmica: faça as perguntas da seção **Relembre** para os alunos e incentive-os a participar e a relembrar os conceitos apresentados nas aulas anteriores. É importante reforçar bem os conceitos aprendidos, pois os alunos precisarão estar com eles bem fixados para as demais aulas.

Expectativas de respostas:
Resposta 1:
O endereço IP é a identidade digital de cada dispositivo, permitindo a comunicação dentro da rede.
Resposta 2:
O IPv4 (32 bits) já está quase esgotado; o IPv6 (128 bits) garante praticamente infinitos endereços.
Resposta 3:
A máscara de sub-rede separa a parte da rede da parte do dispositivo, indicando quem pode se comunicar diretamente.

Slides 7 a 12

Tempo previsto: 20 minutos.

Gestão de sala de aula: inicie a seção **Construindo o conceito** criando um ambiente relaxado e convidativo para um diálogo aberto. Encoraje a participação de todos os estudantes, garantindo que cada voz possa ser ouvida. Caso surjam respostas longas ou debates paralelos, delicadamente redirecione a conversa para o tópico original.

Aprofundamento: o objetivo desta aula é fazer os alunos compreenderem que a internet depende de **protocolos de comunicação**, regras comuns que permitem que diferentes dispositivos e redes funcionem de forma integrada.
O **conjunto TCP/IP** organiza essa comunicação em camadas, distribuindo responsabilidades de modo que a troca de dados ocorra de forma padronizada em qualquer parte do mundo.
A distinção entre **TCP e UDP** mostra que existem abordagens diferentes para atender a necessidades específicas: o TCP garante confiabilidade, reenvio de pacotes e entrega correta, enquanto o UDP prioriza a rapidez, aceitando pequenas perdas em favor da agilidade.
Ao introduzir o **DNS**, a intencionalidade é mostrar como a tecnologia aproxima a complexidade da rede do cotidiano, tornando possível usar nomes simples em vez de longas sequências numéricas.
A conexão desses elementos evidencia que a internet é resultado de um **trabalho conjunto de protocolos**, que equilibra confiabilidade, velocidade e facilidade de uso.

Slides 13 e 14

Orientações: responder ao quiz. Há apenas uma alternativa correta.

Tempo previsto: 2 minutos.

Pergunta: Qual protocolo é usado quando a prioridade é transmitir dados rapidamente, mesmo que alguns pacotes se percam no caminho?
Resposta correta: UDP
Feedback: o UDP prioriza a velocidade da transmissão, sem verificar se todos os pacotes chegaram. O TCP garante confiabilidade, o DNS traduz nomes em endereços e o IP identifica os dispositivos.

Slide 15

Orientações: professor, a seção **Colocando em prática** tem como objetivo aplicar os conhecimentos construídos durante a aula, incentivando os estudantes a pensar criticamente e de forma prática.

Tempo previsto: 18 minutos.

Gestão de sala de aula: Introdução (2 minutos): apresente o cenário: em uma empresa, diferentes serviços dependem de protocolos específicos. A equipe de TI precisa saber identificar **qual protocolo suporta melhor cada necessidade**, garantindo que a comunicação seja eficiente.

Condução da dinâmica:
Atividade em duplas (12 minutos):
Os alunos devem analisar os três cenários propostos e associar cada um ao protocolo correto.
Oriente-os a escrever as justificativas, destacando a **característica central** de cada protocolo (confiabilidade, velocidade, tradução de nomes).
Os alunos devem trabalhar em duplas, discutir as opções e justificar suas respostas.
Circule pela sala para observar e auxiliar nas discussões.
Sorteio e apresentação das respostas (4 minutos):
Escolha duas ou três duplas para compartilharem suas respostas oralmente.

A Atividade deve ser **entregue no AVA** em um local indicado pelo professor.

Expectativas de respostas:
1. **Reunião on-line em vídeo** → **UDP**, porque a prioridade é manter a transmissão fluida e sem atrasos perceptíveis, mesmo com pequenas perdas de pacotes.
2. **Envio de relatórios financeiros** → **TCP**, porque é indispensável que os arquivos cheguem completos, sem falhas ou perdas de informação.
3. **Acesso ao portal da empresa** → **DNS**, porque traduz o nome digitado no navegador no endereço IP do servidor, facilitando o acesso de todos os funcionários.

Slide 16

Orientações: professor, a seção **O que nós aprendemos hoje?** tem como objetivos reforçar e esclarecer os conceitos principais discutidos na aula. Essa revisão pode ser uma ferramenta de avaliação informal do aprendizado dos estudantes, identificando áreas que possam precisar de mais atenção em aulas futuras.

Tempo previsto: 1 minuto.

Gestão de sala de aula:
Mantenha um tom positivo e construtivo, reforçando o aprendizado em vez de focar correções.
Seja direto e objetivo nas explicações, para manter a atividade dentro do tempo estipulado.
Engaje os estudantes rapidamente, pedindo confirmações ou reações breves às definições apresentadas.

Condução da dinâmica:
Explique que esta parte da seção, “Então ficamos assim...”, é um momento de reflexão e de esclarecimento sobre os conceitos abordados na aula.
Informe que será uma rápida revisão para assegurar que os entendimentos dos estudantes estejam alinhados com as definições corretas dos conceitos.
Apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando-o em forma de frases completas.
Destaque se as contribuições dos estudantes estavam alinhadas com os conceitos e ofereça esclarecimentos rápidos caso haja discrepâncias ou mal-entendidos.
Finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.
Reforce a ideia de que essa revisão ajuda a solidificar o entendimento dos estudantes e a prepará-los para aplicar esses conceitos em situações práticas.

Expectativas da atividade:
Os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.
A atividade serve como uma verificação rápida do entendimento dos estudantes e uma oportunidade para corrigir quaisquer mal-entendidos.

Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_ÁREA DO PROFESSOR_**\n",
    "> \n",
    "> **Arquivo:** DADOSANO1C2B1S8A2_PROFESSOR.ipynb\n",
    "> \n",
    "> Este notebook contém a versão do professor para a Aula 2 da Semana 8. Ele inclui todo o conteúdo do aluno, além de orientações, dicas de condução e a solução comentada para cada exercício, formatadas em caixas de citação como esta.\n",
    "> \n",
    "> **Objetivo Geral da Aula Prática:** Levar os alunos a implementar laços com `while` para controlar repetições baseadas em condições lógicas, reforçando a necessidade de uma condição de parada clara para evitar loops infinitos."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">TÉCNICO EM CIÊNCIA DE DADOS</h1>\n",
    "<h1 style=\"text-align: center;\">Roteiro de Atividade Prática - PROFESSOR</h1>\n",
    "
\n",
    "
\n",
    "\n",
    "**Componente:** Programação Aplicada à Ciência de Dados\n",
    "
\n",
    "**Unidade Curricular:** Lógica de Programação e Fundamentos de Python\n",
    "
\n",
    "**Tema da Semana:** Revisão geral e prática de condicionais\n",
    "<br>"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# Aula 2 - Estrutura de repetição while\n",
    "\n",
    "Nesta aula prática, vamos aprender a usar o laço `while`, uma ferramenta poderosa para criar repetições baseadas em condições. Veremos os conceitos, exemplos e resolveremos uma série de exercícios para consolidar o conhecimento."
   ]
  },
  {

"cell_type": "markdown",
"metadata": {},
"source": [
"## Colocando em Prática"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"### Parte 1: Construindo o Conceito - O `while`\n",
"\n",
"O laço `while` é a ferramenta perfeita quando não sabemos quantas vezes uma ação precisa ser repetida. Ele funciona como um \"**`if` que se repete**\".\n",
"\n",
"#### A Sintaxe\n",
"\n",
"A sintaxe é muito parecida com a do `if`:\n",
"```python\n",
"while condicao:\n",
"   # Bloco de código que se repete\n",
"   # ENQUANTO a condição for verdadeira\n",
"```\n",
"- **`condicao`**: É uma expressão que resulta em `True` ou `False`, exatamente como em um `if`.\n",
"- **Bloco de código**: É o conjunto de instruções que será executado repetidamente.\n",
"\n",
"#### O Ponto Mais Importante: A Condição de Parada\n",
"\n",
"Para que um laço `while` não se repita para sempre (um *loop infinito*), algo **dentro** do laço precisa, em algum momento, fazer com que a condição se torne `False`. Geralmente, isso envolve atualizar o valor de uma variável que está sendo testada."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"> **_Para o Professor:_**\n",
"> \n",
"> **Dica de Condução:** Ao explicar a \"Condição de Parada\", reforce veementemente o perigo do *loop infinito*. Você pode até demonstrar rapidamente um código que causa isso (ex: `while True: print(\"Preso!\")`), mostrando como o programa trava e como interromper a execução no ambiente Jupyter (usando o botão de Parar/Stop). Essa demonstração prática tem um forte impacto didático."
]
},
{
"cell_type": "markdown",
"metadata": {},

```
   "source": [
    "### Parte 2: Exemplos Guiados\n",
    "\n",
    "Vamos ver na prática como isso funciona.\n",
    "\n",
    "#### Exemplo 1: Um Contador Simples de 1 a 5\n",
    "\n",
    "Queremos que o programa conte e imprima os números de 1 até 5. Para
isso, precisamos de 3 coisas:\n",
    "1.  **Inicializar** uma variável contadora (começando em 1).\n",
    "2.  Criar um laço `while` que continue **enquanto** o contador for
menor ou igual a 5.\n",
    "3.  **Atualizar** o contador a cada ciclo, somando 1 a ele."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 1,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "1\n",
      "2\n",
      "3\n",
      "4\n",
      "5\n",
      "Laço finalizado!\n"
     ]
    }
   ],
   "source": [
    "# 1. Inicialização\n",
    "contador = 1\n",
    "\n",
    "# 2. Condição\n",
    "while contador <= 5:\n",
    "    print(contador)\n",
    "    # 3. Atualização (essencial para não criar um loop
infinito!)\n",
    "    contador = contador + 1\n",
    "\n",
    "print(\"Laço finalizado!\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Para o Professor:_**\n",
    "> \n",
```

"> **Dica de Condução:** Ao apresentar este exemplo, narre a execução passo a passo, como se você fosse o computador: \"O contador começa valendo 1. 1 é menor ou igual a 5? Sim. Imprimo 1 e somo 1 ao contador. Agora ele vale 2. 2 é menor ou igual a 5? Sim...\" e assim por diante, até o momento em que o contador vira 6 e a condição se torna falsa."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "#### Exemplo 2: O Validador de Senha\n",
    "\n",
    "Agora, o problema que discutimos: pedir uma senha até que o usuário acerte.\n",
    "\n",
    "1. **Definir** a senha secreta.\n",
    "2. **Pedir** ao usuário para digitar uma senha pela primeira vez.\n",
    "3. Criar um laço `while` que continue **enquanto** a senha digitada for diferente da senha secreta.\n",
    "4. Dentro do laço, avisar que a senha está errada e **pedir novamente**."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 2,
   "metadata": {},
   "outputs": [
    {
     "name": "stdin",
     "output_type": "stream",
     "text": [
      "Digite a senha: 235\n"
     ]
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Senha incorreta. Tente novamente.\n"
     ]
    },
    {
     "name": "stdin",
     "output_type": "stream",
     "text": [
      "Digite a senha: 6\n"
     ]
    },
    {
     "name": "stdout",
     "output_type": "stream",

```
      "text": [
       "Senha incorreta. Tente novamente.\n"
      ]
     },
     {
      "name": "stdin",
      "output_type": "stream",
      "text": [
       "Digite a senha:  546456\n"
      ]
     },
     {
      "name": "stdout",
      "output_type": "stream",
      "text": [
       "Senha incorreta. Tente novamente.\n"
      ]
     },
     {
      "name": "stdin",
      "output_type": "stream",
      "text": [
       "Digite a senha:  1234\n"
      ]
     },
     {
      "name": "stdout",
      "output_type": "stream",
      "text": [
       "Acesso permitido!\n"
      ]
     }
    ],
    "source": [
     "senha_secreta = \"1234\"\n",
     "\n",
     "# Pedimos a senha uma vez, fora do laço\n",
     "senha_digitada = input(\"Digite a senha: \")\n",
     "\n",
     "# O laço só começa se a primeira tentativa estiver errada\n",
     "while senha_digitada != senha_secreta:\n",
     "    print(\"Senha incorreta. Tente novamente.\")\n",
     "    # Pedimos a senha novamente. Esta é a atualização da
variável!\n",
     "    senha_digitada = input(\"Digite a senha: \")\n",
     "\n",
     "# Este código só será executado quando o laço terminar\n",
     "print(\"Acesso permitido!\")"
    ]
   },
   {
    "cell_type": "markdown",
    "metadata": {},
    "source": [
```

"---\n",
"### Parte 3: Exercícios Práticos\n",
"\n",
"Agora é a sua vez! Resolva os exercícios abaixo, focando na criação da **condição de parada** correta para cada laço `while`.\n",
"\n",
"**Importante:** Este é nosso primeiro contato prático com o `while`. Aprofundaremos seu uso e aplicações na **Semana 10**."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"#### Exercício 1: Contagem Regressiva\n",
"Crie um programa que faça uma contagem regressiva de 10 até 1 e, no final, exiba a mensagem \"Fogo!\"."
]
},
{
"cell_type": "code",
"execution_count": 3,
"metadata": {},
"outputs": [],
"source": [
"# Escreva sua solução aqui\n",
"\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"> **_Para o Professor - Exercício 1:_**\n",
"> \n",
"> **Objetivo da Atividade:** Praticar a estrutura básica do `while` (inicialização, condição, atualização) com uma lógica de decremento em vez de incremento.\n",
"> \n",
"> **Ponto de Atenção:** O erro mais comum será esquecer de subtrair 1 do contador a cada passo, causando um loop infinito. Verifique se os alunos implementaram a linha `contador = contador - 1` (ou `contador -= 1`).\n",
"> \n",
"> ---\n",
"> \n",
"> **_Solução Comentada:_**"
]
},
{
"cell_type": "code",
"execution_count": 4,
"metadata": {},
"outputs": [

```
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "10\n",
      "9\n",
      "8\n",
      "7\n",
      "6\n",
      "5\n",
      "4\n",
      "3\n",
      "2\n",
      "1\n",
      "Fogo!\n"
     ]
    }
   ],
   "source": [
    "# Inicializamos o contador com o valor de partida\n",
    "numero = 10\n",
    "\n",
    "# A condição continua enquanto o número for maior ou igual a 1\n",
    "while numero >= 1:\n",
    "    print(numero)\n",
    "    # A atualização agora subtrai 1\n",
    "    numero = numero - 1\n",
    "\n",
    "print(\"Fogo!\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "---\n",
    "#### Exercício 2: Adivinhe o Número\n",
    "Crie um jogo simples onde o computador \"pensa\" em um número
secreto (neste caso, vamos defini-lo como 42). O programa deve pedir ao
usuário para adivinhar o número. Enquanto o palpite do usuário estiver
errado, o programa deve dizer \"Errado, tente de novo!\" e pedir um novo
palpite. Quando o usuário acertar, o programa deve exibir \"Parabéns,
você acertou!\"."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 5,
   "metadata": {},
   "outputs": [],
   "source": [
    "# Escreva sua solução aqui\n",
    "\n"
   ]
```

```
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Para o Professor - Exercício 2:_**\n",
    "> \n",
    "> **Objetivo da Atividade:** Reforçar a lógica de repetição baseada
na entrada do usuário, similar ao exemplo da senha, mas com dados
numéricos.\n",
    "> \n",
    "> **Ponto de Atenção:** O erro mais provável é esquecer de converter
o palpite do usuário (`input`) para um número inteiro (`int`) antes de
compará-lo com o número secreto. Isso gerará um erro de lógica, pois a
string \"42\" é diferente do número 42.\n",
    "> \n",
    "> ---\n",
    "> \n",
    "> **_Solução Comentada:_**"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 6,
   "metadata": {},
   "outputs": [
    {
     "name": "stdin",
     "output_type": "stream",
     "text": [
      "Adivinhe o número secreto:  50\n"
     ]
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Errado, tente de novo!\n"
     ]
    },
    {
     "name": "stdin",
     "output_type": "stream",
     "text": [
      "Adivinhe o número secreto:  42\n"
     ]
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Parabéns, você acertou!\n"
     ]
    }
```

```
   ],
   "source": [
    "numero_secreto = 42\n",
    "\n",
    "# Pedimos o palpite e o convertemos para inteiro\n",
    "palpite = int(input(\"Adivinhe o número secreto: \"))\n",
    "\n",
    "# A condição para o laço continuar é o palpite ser diferente do
número secreto\n",
    "while palpite != numero_secreto:\n",
    "    print(\"Errado, tente de novo!\")\n",
    "    # A atualização é pedir um novo palpite ao usuário\n",
    "    palpite = int(input(\"Adivinhe o número secreto: \"))\n",
    "\n",
    "print(\"Parabéns, você acertou!\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "---\n",
    "#### Exercício 3: Menu Interativo\n",
    "Crie um programa que exibe um menu simples com as opções: 1. Ver
saldo, 2. Fazer depósito, 3. Sair. O programa deve continuar exibindo o
menu e pedindo uma opção ao usuário enquanto a opção digitada não for
\"3\" (a opção de sair). Para as opções 1 e 2, apenas exiba uma mensagem
simples como \"Exibindo saldo...\" ou \"Realizando depósito...\"."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 7,
   "metadata": {},
   "outputs": [],
   "source": [
    "# Escreva sua solução aqui\n",
    "\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Para o Professor - Exercício 3:_**\n",
    "> \n",
    "> **Objetivo da Atividade:** Praticar a criação de um loop
controlado por uma opção de \"sair\" e o uso de `if-elif` dentro do
`while` para tratar diferentes ações do usuário.\n",
    "> \n",
    "> **Dica de Condução:** Mostre como a variável `opcao` é
inicializada com um valor qualquer (como \"0\") para garantir que o laço
`while` execute pelo menos uma vez. Esta é uma técnica comum para menus
interativos.\n",
```

```
  "> \n",
  "> ---\n",
  "> \n",
  "> **_Solução Comentada:_**"
 ]
},
{
 "cell_type": "code",
 "execution_count": 8,
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "\n",
    "--- MENU ---\n",
    "1. Ver saldo\n",
    "2. Fazer depósito\n",
    "3. Sair\n"
   ]
  },
  {
   "name": "stdin",
   "output_type": "stream",
   "text": [
    "Escolha uma opção:  1\n"
   ]
  },
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "Exibindo saldo...\n",
    "\n",
    "--- MENU ---\n",
    "1. Ver saldo\n",
    "2. Fazer depósito\n",
    "3. Sair\n"
   ]
  },
  {
   "name": "stdin",
   "output_type": "stream",
   "text": [
    "Escolha uma opção:  2\n"
   ]
  },
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "Realizando depósito...\n",
    "\n",
```

"--- MENU ---\n",
"1. Ver saldo\n",
"2. Fazer depósito\n",
"3. Sair\n"
]
},
{
"name": "stdin",
"output_type": "stream",
"text": [
"Escolha uma opção: 3\n"
]
},
{
"name": "stdout",
"output_type": "stream",
"text": [
"Saindo do sistema. Até logo!\n"
]
}
],
"source": [
"opcao = \"0\" # Inicializa a variável para garantir que o laço comece\n",
"\n",
"# O laço continua enquanto a opção não for '3'\n",
"while opcao != \"3\":\n",
"   print(\"\\n--- MENU ---\")\n",
"   print(\"1. Ver saldo\")\n",
"   print(\"2. Fazer depósito\")\n",
"   print(\"3. Sair\")\n",
"   \n",
"   # A atualização da variável 'opcao' ocorre a cada ciclo\n",
"   opcao = input(\"Escolha uma opção: \")\n",
"\n",
"   if opcao == \"1\":\n",
"   print(\"Exibindo saldo...\")\n",
"   elif opcao == \"2\":\n",
"   print(\"Realizando depósito...\")\n",
"   elif opcao == \"3\":\n",
"   print(\"Saindo do sistema. Até logo!\")\n",
"   else:\n",
"   print(\"Opção inválida. Tente novamente.\")"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"---\n",
"#### Exercício 4 (Desafio): Somando até Parar\n",
"Crie um programa que peça ao usuário para digitar números inteiros um após o outro. O programa deve somar todos os números digitados. A repetição deve parar quando o usuário digitar o número 0. No final, o

programa deve exibir a soma total dos números inseridos (sem incluir o 0)."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 9,
   "metadata": {},
   "outputs": [],
   "source": [
    "# Escreva sua solução aqui\n",
    "\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Para o Professor - Exercício 4:_**\n",
    "> \n",
    "> **Objetivo da Atividade:** Introduzir a ideia de um \"acumulador\" (`soma`) que é atualizado a cada ciclo, combinado com uma condição de parada baseada em um valor sentinela (o número 0).\n",
    "> \n",
    "> **Ponto de Atenção:** A lógica pode ser desafiadora. O `input` precisa estar dentro do laço para pedir novos números, e a soma (`soma = soma + numero`) também. O `print` do resultado final deve estar fora do laço para exibir apenas o total.\n",
    "> \n",
    "> ---\n",
    "> \n",
    "> **_Solução Comentada:_**"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 10,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Digite números para somar. Digite 0 para parar.\n"
     ]
    },
    {
     "name": "stdin",
     "output_type": "stream",
     "text": [
      "Digite um número: 10\n",
      "Digite um número: 20\n",
      "Digite um número: 10\n",
      "Digite um número: 0\n"

```
     ]
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "A soma total dos números digitados é: 40\n"
     ]
    }
   ],
   "source": [
    "# Inicializamos o acumulador da soma e a variável que guardará o
número\n",
    "soma_total = 0\n",
    "numero_digitado = -1 # Qualquer valor diferente de 0 para iniciar o
laço\n",
    "\n",
    "print(\"Digite números para somar. Digite 0 para parar.\")\n",
    "\n",
    "while numero_digitado != 0:\n",
    "    # Pedimos um número ao usuário a cada ciclo\n",
    "    numero_digitado = int(input(\"Digite um número: \"))\n",
    "    \n",
    "    # Adicionamos o número à soma total\n",
    "    soma_total = soma_total + numero_digitado\n",
    "\n",
    "print(f\"A soma total dos números digitados é: {soma_total}\")"
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3 (ipykernel)",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.8"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 4
}
```

Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

Introdução à informática

Endereçamento e protocolos de rede

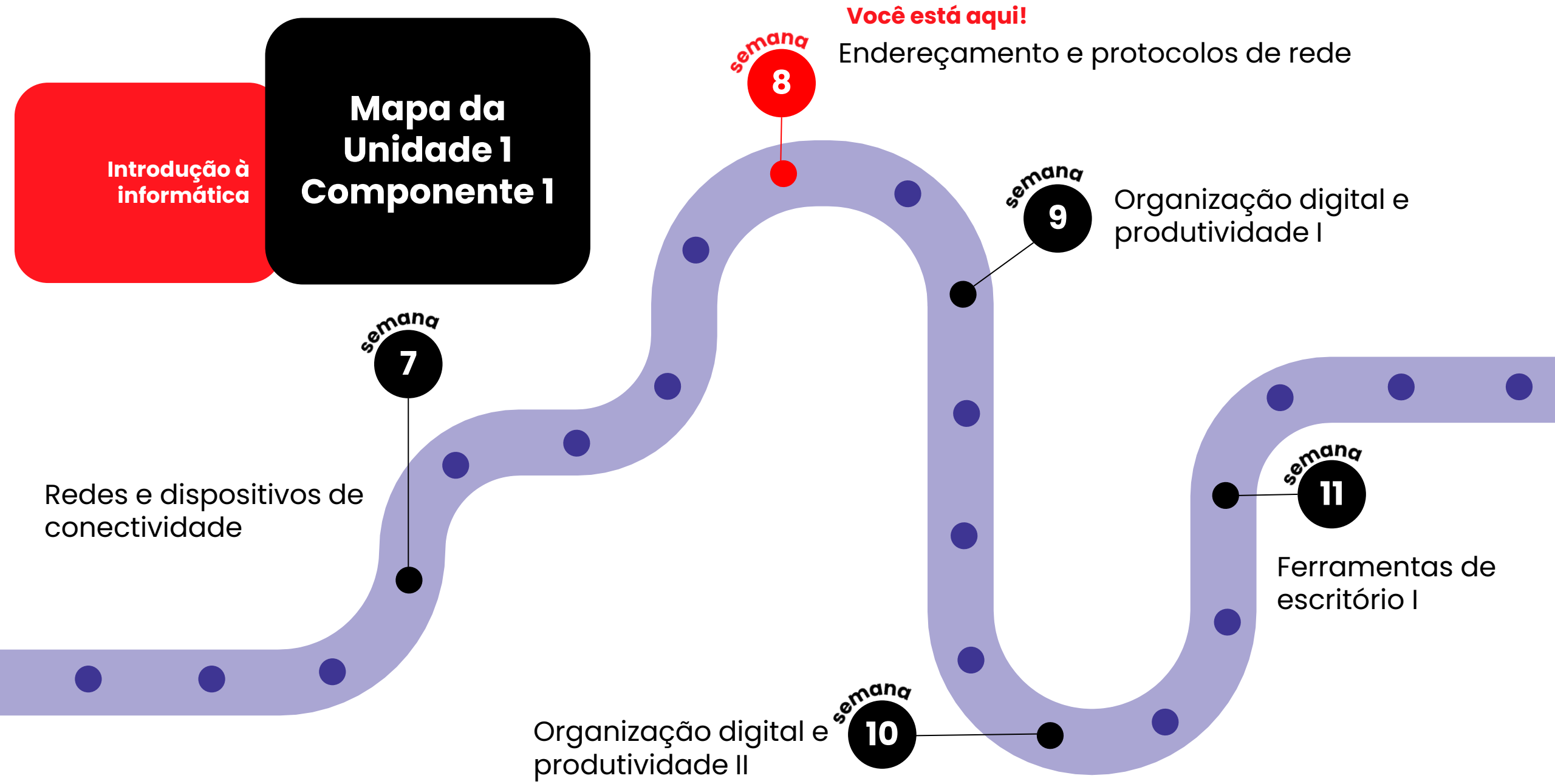
Aula 3: Conectando e diagnosticando redes

Código da aula: DADOSANO1C1B2S8A3

GOVERNO DO ESTADO
DE SÃO PAULO

Introdução à informática

Mapa da Unidade 1 Componente 1



Introdução à informática

Mapa da Unidade 1 Componente 1

Você está aqui!

Endereçamento e protocolos de rede

8

Aula 3: Conectando e diagnosticando redes

Código da aula: DADOSANO1C1B2S8A3

Objetivos da aula

- Compreender e aplicar o conceito de conectando e diagnosticando redes.

Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e/ou à internet.
- Papel e caneta.

Duração da aula

50 minutos.

Habilidades técnicas

- Interpretar endereçamento IP e funcionamento de protocolos de rede.

Habilidades socioemocionais

- Interagir com confiança e disposição para aprender com os outros.

Relembre

Protocolos de rede

- O que são protocolos de rede e por que são importantes em uma empresa?
- Qual é a diferença entre TCP e UDP?
- Para que serve o DNS no dia a dia?

Construindo o **conceito**

Detectando problemas na rede

Diagnóstico de rede é o processo de verificar se os dispositivos estão conectados e se a comunicação está fluindo. É importante para:

- descobrir **se o problema está no computador, no roteador ou na internet**;
- investigar onde está o problema da conexão e identificar rapidamente falhas, usando ferramentas simples como **ping, tracert, ipconfig e status de rede**.

Exemplo: se o YouTube não abre, o diagnóstico mostra se a falha está na sua máquina, no Wi-Fi ou no site.

Fonte: STALLINGS, 2017.

Construindo o **conceito**

Ping: testando a conexão

Ping (Packet Internet Groper) é a ferramenta mais simples e acessível para checar se a internet está funcionando. Esse comando envia um pacote para outro dispositivo e verifica se ele responde, mostrando se há **conexão** e medindo o **tempo de resposta (latência)**.

Prompt de Comando

```
Microsoft Windows [versão 10.0.26100.6584]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\mendo> ping www.google.com

Disparando www.google.com [142.250.219.36] com 32 bytes de dados:
Resposta de 142.250.219.36: bytes=32 tempo=18ms TTL=251
Resposta de 142.250.219.36: bytes=32 tempo=19ms TTL=251
Resposta de 142.250.219.36: bytes=32 tempo=17ms TTL=251
Resposta de 142.250.219.36: bytes=32 tempo=17ms TTL=251

Estatísticas do Ping para 142.250.219.36:
    Pacotes: Enviados = 4, Recebidos = 4, Perdidos = 0 (0% de
             perda),
Aproximar um número redondo de vezes em milissegundos:
    Mínimo = 17ms, Máximo = 19ms, Média = 17ms
```

Produzido pela SEDUC-SP com a ferramenta Microsoft Windows.

Como usar (Windows):

1. Abra o **Prompt de Comando**.
2. Digite: ping www.google.com
3. Se houver resposta, o PC está conectado.

Nesse exemplo, o **ping** retornou a resposta em 17 ms de média, isso significa que a internet está ativa e estável.

Fonte: BROOKSHEAR, 2013.

Construindo o **conceito**

Tracert: o caminho dos dados

Tracert (Traceroute em Linux/Mac) é o comando que mostra **por onde os pacotes passam** até chegarem **ao destino**. Ele lista todos os **roteadores e servidores intermediários**, ajudando a identificar onde há falha ou lentidão no caminho.

Prompt de Comando

```
C:\Users\mendo> tracert www.youtube.com

Rastreando a rota para youtube-ui.l.google.com [142.251.132.238]
com no máximo 30 saltos:

  1     1 ms    <1 ms     2 ms  192.168.0.1
  2    19 ms    10 ms     9 ms  100.72.34.1
  3     3 ms     4 ms    15 ms  bd3db0a5.virtua.com.br [189.61.176.165]
  4     *        *        *     Esgotado o tempo limite do pedido.
  5     5 ms     5 ms     6 ms  bd3db1f6.virtua.com.br [189.61.177.246]
  6     8 ms    11 ms    12 ms  bd3db1f5.virtua.com.br [189.61.177.245]
  7     9 ms     5 ms     5 ms  189.53.92.65
  8     *        *        *     Esgotado o tempo limite do pedido.
  9     *        *        *     Esgotado o tempo limite do pedido.
 10     *       16 ms    14 ms  200.244.18.25
 11    13 ms    27 ms    15 ms  200.244.19.96
 12    15 ms    14 ms    15 ms  peer-B54-agg04.rjoen.embratel.net.br [200.242.4.2]
 13    95 ms    41 ms    21 ms  142.250.228.55
 14    15 ms    36 ms    13 ms  216.239.54.237
 15    15 ms    35 ms    14 ms  lcrioa-am-in-f14.1e100.net [142.251.132.238]

Rastreamento concluído.
```

Produzido pela SEDUC-SP com a ferramenta Microsoft Windows.

Como usar (Windows):

Abra o **Prompt de Comando**.

1. Digite: tracert www.youtube.com
2. Veja cada “salto” até o servidor.

Se o **tracert** parar antes de chegar ao destino final, pode indicar falha no caminho. Mas se ele mostrar mensagens de tempo esgotado em alguns pontos e ainda assim alcançar o destino, significa que houve apenas bloqueio de resposta nesses trechos, não uma falha completa da conexão.

Fonte: BROOKSHEAR, 2013.

Construindo o **conceito**

Descobrindo o seu IP

IP (Internet Protocol) é o número que identifica cada dispositivo em uma rede. Sem IP válido, o computador não consegue se comunicar.

Como descobrir o IP do seu dispositivo?

1. Windows: ipconfig
2. Linux/Mac: ifconfig ou ip addr
3. Celulares: Configurações > Wi-Fi > detalhes da rede
4. Navegador: Pesquise no Google: *"qual é o meu IP"*.

Fonte: MONTEIRO, 2019.

PARA REFLETIR

Por que isso é importante para um cientista de dados?

- ✓ Permite **diagnosticar conexões** em projetos que dependem da internet.
- ✓ Ajuda a **entender logs de sistemas e servidores**, que registram endereços IP.
- ✓ Viabiliza o **trabalho com Big Data em redes**, pois cada máquina conectada gera dados que precisam ser identificados.

Construindo o **conceito**

Status de rede

Ícone normal de Wi-Fi

- Significa que o dispositivo está conectado ao roteador e à internet.
- Permite acessar sites, assistir a vídeos e jogar on-line sem problemas.

Ícone de Wi-Fi com um “!”

- Mostra que o dispositivo se conecta ao roteador, mas não chega até a internet.
- É como falar com o porteiro do prédio, mas não conseguir sair para a rua.

Ícone de cabo com “X” ou Wi-Fi sem sinal

- Significa que o dispositivo não tem conexão com a rede.
- É como se o cabo estivesse desconectado ou o Wi-Fi desligado.

Imagens: © Getty Images

Registro

Pause e **responda**

Você não consegue acessar um site no computador. Qual comando deve usar primeiro para verificar se há conexão básica com a internet?

- status de rede
- ipconfig
- ping
- tracert

Registro

Pause e
responda

Você não consegue acessar um site no computador. Qual comando deve usar primeiro para verificar se há conexão básica com a internet?

- ✗ status de rede
- ✗ ipconfig
- ✓ ping
- ✗ tracert

Ser
sempre +

Situação

Durante a manhã, o laboratório de informática da escola apresenta lentidão na conexão com a internet.

Alguns computadores navegam normalmente, mas outros não conseguem acessar nenhuma página.

© Getty Images

A equipe precisa **analisar o problema em conjunto**, mantendo uma postura colaborativa e investigativa para identificar onde pode estar a causa da falha.

Situação fictícia produzida pela SEDUC-SP.

Ser
sempre +

VIREM E CONVERSEM

Ação

Em duplas:

- Quais **evidências vocês buscariam** para identificar a origem do problema?
- Como poderiam **trabalhar em equipe**, distribuindo as tarefas e comunicando os resultados de forma organizada?

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Diagnosticar redes significa identificar onde está a falha: no dispositivo, no roteador ou na internet.

2. Ferramentas como ping, tracert e a verificação de IP ajudam a testar e a localizar problemas de conexão.

3. Os ícones de status da rede oferecem sinais visuais que permitem interpretar rapidamente a situação da conexão.

Saiba mais

Quer entender mais sobre diagnóstico de rede?

Recomendamos que assista ao vídeo:

THE TELECOM. Teste de ping, ipconfig e tracert verificando sua conexão. YouTube, 6 abr. 2021. Disponível em: https://www.youtube.com/watch?v=MhXnEhvgtf4. Acesso em: 14 out. 2025.

Referências da aula

BROOKSHEAR, J. G. **Ciência da computação**: uma visão abrangente. Porto Alegre: Bookman, 2013.

MONTEIRO, M. A. **Introdução à organização de computadores**. Rio de Janeiro: LTC, 2019.

STALLINGS, W. **Arquitetura e organização de computadores**. São Paulo: Pearson Education do Brasil, 2017.

TANENBAUM, A. S.; AUSTIN, T. **Organização estruturada de computadores**. São Paulo: Pearson Prentice Hall, 2013.

Identidade visual: imagens © Getty Images

Orientações ao professor

Slide 5

Tempo previsto:
Relembre: 8 minutos
Construindo o conceito: 20 minutos
Pause e responda: 2 minutos
Ser sempre +: 18 minutos
O que nós aprendemos hoje?: 1 minuto
Saiba mais: 1 minuto

Slide 6

Tempo previsto: 8 minutos.

Gestão de sala de aula: assegure que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos, para garantir a participação de todos. Mantenha um ambiente de respeito, no qual todas as opiniões sejam valorizadas, garantindo que todos se sintam confortáveis para expressar seus pontos de vista.

Condução da dinâmica: faça as perguntas da seção **Relembre** para os alunos e incentive-os a participar e a relembrar os conceitos apresentados nas aulas anteriores. É importante reforçar bem os conceitos aprendidos, pois os alunos precisarão estar com eles bem fixados para as demais aulas.

Expectativas de respostas:
Resposta 1: Protocolos de rede são regras que permitem que diferentes dispositivos conversem de forma organizada, garantindo comunicação segura e eficiente em qualquer ambiente corporativo.
Resposta 2: TCP prioriza confiabilidade e entrega completa (ex.: envio de arquivos e documentos importantes). **UDP** prioriza velocidade e fluidez (ex.: reuniões on-line ou transmissões em tempo real).
Resposta 3: O **DNS** funciona como um tradutor que transforma nomes fáceis em endereços IP, tornando prático acessar sistemas ou sites corporativos sem precisar memorizar números.

Slides 7 a 11

Tempo previsto: 20 minutos.

Gestão de sala de aula: inicie a seção **Construindo o conceito** criando um ambiente relaxado e convidativo para um diálogo aberto. Encoraje a participação de todos os estudantes, garantindo que cada voz possa ser ouvida. Caso surjam respostas longas ou debates paralelos, delicadamente redirecione a conversa para o tópico original.

Aprofundamento: o objetivo desta aula é mostrar que, antes de resolver um problema de rede, é preciso **diagnosticar onde está a falha**. A intenção pedagógica é fazer o aluno perceber que existem ferramentas simples, acessíveis e já disponíveis em qualquer sistema operacional, que permitem identificar se o problema está no próprio dispositivo, no roteador ou na internet.
O **ping** é apresentado como o primeiro teste: envia uma pequena mensagem e aguarda resposta, permitindo verificar se outro dispositivo está ativo. O **tracert (ou traceroute)** aprofunda essa análise, revelando o caminho que os pacotes percorrem até o destino e ajudando a identificar onde pode estar a lentidão ou a perda de conexão. A verificação do **endereço IP** mostra se o computador recebeu um número válido na rede, reforçando a importância do endereçamento para a comunicação.
Os **ícones de status da rede** aproximam ainda mais o conteúdo da realidade do aluno, pois são sinais visuais que ele já encontra no dia a dia, mas que agora aprende a interpretar de forma técnica.
Com isso, os estudantes compreendem que **diagnosticar redes é mais do que "tentar reconectar"**: é seguir um raciocínio lógico, utilizando ferramentas básicas para localizar e entender os problemas. A aula fortalece o pensamento crítico e prepara para situações práticas que encontrarão tanto em casa quanto em ambientes profissionais.

Slides 12 e 13

Orientações: responder ao quiz. Há apenas uma alternativa correta.

Tempo previsto: 2 minutos.

Pergunta: Você não consegue acessar um site no computador. Qual comando deve usar primeiro para verificar se há conexão básica com a internet?
Resposta correta: ping
Feedback: o comando **ping** é o primeiro teste para verificar se existe conexão. O **ipconfig** mostra o IP do dispositivo, o **tracert** revela o caminho dos pacotes e o **status** indica apenas sinais visuais da rede.

Slides 14 e 15

Orientações: a seção **Ser sempre +** tem como objetivos desenvolver e aprimorar as competências socioemocionais dos estudantes, focando especificamente nas situações desafiadoras que podem surgir no ambiente profissional.

Tempo previsto: 18 minutos.

Gestão de sala de aula: Introdução (3 minutos): apresente a situação de maneira simples e direta. Após apresentar, organize os estudantes em duplas.

Condução da dinâmica:
Atividade em duplas:
Planejamento inicial (10 minutos):
Instruções: em duplas, discutam como podem lidar com essa situação.

Apresentação e discussão (5 minutos):
Apresentação rápida: sortear algumas duplas para apresentar suas principais ideias e soluções para a turma. Cada dupla tem 2 minutos para compartilhar suas sugestões de como lidar com a situação.

Expectativas de respostas: sugerir verificações como conflitos de IP, configuração de gateway, teste de ping, ou falhas no DHCP.

Propor uma **comunicação estruturada entre colegas**: respeitar opiniões divergentes e organizar o raciocínio conjunto. Compreender que tanto o domínio técnico quanto a **cooperação entre profissionais** são fundamentais para solucionar problemas em redes de computadores.

Slide 16

Orientações: professor, a seção **O que nós aprendemos hoje?** tem como objetivos reforçar e esclarecer os conceitos principais discutidos na aula. Essa revisão pode ser uma ferramenta de avaliação informal do aprendizado dos estudantes, identificando áreas que possam precisar de mais atenção em aulas futuras.

Tempo previsto: 1 minuto.

Gestão de sala de aula:
Mantenha um tom positivo e construtivo, reforçando o aprendizado em vez de focar correções.
Seja direto e objetivo nas explicações, para manter a atividade dentro do tempo estipulado.
Engaje os estudantes rapidamente, pedindo confirmações ou reações breves às definições apresentadas.

Condução da dinâmica:
Explique que esta parte da seção, "Então ficamos assim...", é um momento de reflexão e de esclarecimento sobre os conceitos abordados na aula.
Informe que será uma rápida revisão para assegurar que os entendimentos dos estudantes estejam alinhados com as definições corretas dos conceitos.
Apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando-o em forma de frases completas.
Destaque se as contribuições dos estudantes estavam alinhadas com os conceitos e ofereça esclarecimentos rápidos caso haja discrepâncias ou mal-entendidos.
Finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.
Reforce a ideia de que essa revisão ajuda a solidificar o entendimento dos estudantes e a prepará-los para aplicar esses conceitos em situações práticas.

Expectativas da atividade:
Os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.
A atividade serve como uma verificação rápida do entendimento dos estudantes e uma oportunidade para corrigir quaisquer mal-entendidos.

Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de Python

Revisão geral e prática de condicionais

Aula 4: Exercícios com for e while em diferentes contextos

Código da aula: DADOSANO1C2B2S8A4

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de Python

Mapa da Unidade 1 Componente 2

semana 5

Operadores lógicos

semana 6

Estruturas condicionais: if, else

semana 7

Estruturas condicionais: if-elif-else

semana 8

Você está aqui!

Revisão geral e prática de condicionais

semana 9

Laços de repetição: for

Lógica de programação e fundamentos de Python

Mapa da Unidade 1 Componente 2

Você está aqui!

Revisão geral e prática de condicionais

8

Aula 4: Exercícios com for e while em diferentes contextos

Código da aula: DADOSANO1C2B2S8A4

Objetivos da aula

- Utilizar estruturas de repetição em desafios contextualizados com foco em autonomia.

Recursos didáticos

- Roteiro de Atividade Prática.
- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e à internet.
- Ferramentas de codificação para Python.

Duração da aula

50 minutos.

Habilidades técnicas

- Implementar laços de repetição utilizando for e while para automatizar tarefas repetitivas.

Habilidades socioemocionais

- Assumir compromissos com autonomia e responsabilidade.

Relembre

Comandos de controle

- Qual é a função do comando break dentro de um laço de repetição?
- Quando usamos o comando continue dentro de um laço?

Colocando em **prática**

Registro

Exercícios com for e while em diferentes contextos

Prática guiada

Individual

40 minutos

Durante a aula

Envio ao AVA

Materiais necessários

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática.
- Software Jupyter Notebook ou Google Colab.

Passo a passo

1. Baixe o arquivo “DADOSANO1C2B1S7A4_ALUNO.ipynb”; nele já contém o roteiro necessário.
2. Execute cada passo em conjunto com o professor.
3. Resolva as atividades propostas.
4. Faça a entrega no AVA.

Baixe o roteiro dessa atividade / Baixe o material de apoio dessa atividade

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Vimos o laço certo para cada tipo de repetição, analisando a condição de controle.

2. Aprendemos como combinar for, while, if e break para criar soluções completas em código.

3. Desenvolvemos raciocínio lógico e autonomia ao resolver desafios com múltiplas condições de parada.

Saiba mais

Praticando Python: laços for e while

Este artigo do blog da Alura detalha ainda mais as diferenças entre os laços for e while em Python. Ele apresenta outros exemplos e contextos, ajudando você a solidificar sua intuição sobre quando cada laço é a opção mais elegante e eficiente.

Leia:

SANTANA, A. Praticando Python: laços for e while. **Alura**, [s.d.]. Disponível em: https://www.alura.com.br/conteudo/praticando-python-lacos-for-while. Acesso em: 27 out. 2025.

Referências da aula

LUTZ, M. **Learning Python**. Sebastopol: O'Reilly Media, 2013.

MENEZES, N. N. C. **Introdução à programação com Python**: algoritmos e lógica de programação para iniciantes. São Paulo: Novatec, 2019.

PYTHON. **Documentação Python 3.14.0**, [s.d.]. Disponível em: https://docs.python.org/pt-br/3/. Acesso em: 27 out. 2025.

Identidade visual: imagens © Getty Images

Orientações ao professor

Slide 6

Orientações: a **seção Relembre** tem o objetivo de retomar conhecimentos já adquiridos por meio de atividades ou exercícios simples, reforçando conceitos previamente trabalhados.

Tempo previsto: 7 minutos.

Gestão de sala de aula:
Assegure que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos para garantir a participação de todos.
Mantenha um ambiente de respeito, em que todas as opiniões são valorizadas, garantindo que os alunos se sintam confortáveis para expressar seus pontos de vista.

Condução da dinâmica:
Faça as perguntas dessa seção para os alunos e incentive-os a participar e relembrar os conceitos apresentados nas aulas anteriores.
É importante reforçar os conceitos aprendidos, pois os alunos precisarão fixá-los bem para as demais aulas.

Expectativas de respostas:
Resposta 1: O break interrompe o laço imediatamente, mesmo que ainda existam elementos para percorrer.
Resposta 2: O continue é usado para pular uma iteração específica e seguir para a próxima repetição sem encerrar o laço.

Slide 7

Orientações: a **seção Colocando em prática** tem como objetivo aplicar os conhecimentos construídos durante a aula, incentivando os estudantes a pensarem criticamente e de forma prática.

Tempo previsto: 40 minutos.

Gestão de sala de aula:
Assegure que todos os alunos estejam com o arquivo **DADOSANO1C2B2S8A4_ALUNO.ipynb** aberto e pronto para a execução.
Incentive o trabalho em duplas. A resolução dos desafios, especialmente os finais, beneficia-se da discussão sobre qual estrutura de laço é mais apropriada para cada cenário.
Circule ativamente pela sala, agindo como um consultor. Em vez de dar a resposta, faça perguntas que guiem o raciocínio: "Este problema tem um número fixo de repetições ou depende de uma condição?", "Qual variável vai controlar a parada do seu laço while?".

Condução da dinâmica:
Inicie a seção revisando rapidamente com a turma a "Parte 1: Revisando nossas ferramentas". Peça aos alunos que leiam as dicas e reforce a importância de analisar o problema antes de começar a codificar.
Libere os alunos para resolverem os exercícios em seu próprio ritmo. A progressão de dificuldade foi projetada para construir a confiança.
Após a maioria ter concluído o "Exercício 3", faça uma pausa para uma rápida verificação de aprendizado. Peça a um aluno que explique por que usou o for e a outro que explique por que usou o while no exercício anterior.
Dê um tempo maior para os desafios 4 and 5. O "Desafio 5" é particularmente importante, pois pode ser resolvido de mais de uma maneira (for ou while). Se o tempo permitir, convide duas duplas para apresentarem as diferentes soluções no projetor e discuta os prós e contras de cada abordagem com a turma.

Expectativas de respostas:
Professor, os alunos devem entregar no AVA o resultado da atividade prática.
Espera-se que os alunos consigam analisar cada problema e justificar a escolha da estrutura de repetição (for para repetições contadas, while para repetições condicionais).
As soluções devem ser funcionais e demonstrar o uso correto de input, operadores, condicionais e, quando necessário, comandos de controle como o break.
O gabarito completo e comentado para todos os exercícios, incluindo as duas abordagens para o desafio final, está disponível no arquivo do professor: **DADOSANO1C2B2S8A4_PROFESSOR.ipynb**.

Slide 8

Orientações: a **seção O que nós aprendemos hoje?** tem o objetivo de reforçar e esclarecer os conceitos principais discutidos na aula. Essa dinâmica pode ser uma ferramenta de avaliação informal do aprendizado dos estudantes, identificando áreas que podem precisar de mais atenção em aulas futuras.

Tempo previsto: 2 minutos.

Gestão de sala de aula:
Mantenha um tom positivo e construtivo, reforçando o aprendizado em vez de focar correções.
Seja direto e objetivo nas explicações para manter a atividade de acordo com o tempo estipulado.
Engaje os estudantes, rapidamente, pedindo confirmações ou reações breves às definições apresentadas.

Condução da dinâmica:
Explique que esta parte da seção, Então ficamos assim..., é um momento de reflexão e esclarecimento sobre os conceitos abordados na aula.
Informe que será realizada uma rápida revisão para assegurar que os entendimentos dos estudantes estão alinhados às definições corretas dos conceitos.
Apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando em forma de frases completas.
Destaque se as contribuições dos estudantes estão alinhadas ao conceito e ofereça esclarecimentos rápidos caso haja discrepâncias ou mal-entendidos.
Finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.
Reforce a ideia de que essa revisão ajuda a solidificar o entendimento dos estudantes e prepará-los para aplicar esses conceitos em situações práticas.

Expectativas de respostas:
Os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.
A atividade serve como uma verificação rápida do entendimento dos estudantes e uma oportunidade para corrigir quaisquer mal-entendidos.

Slide 9

Orientações: a seção **Saiba Mais+** tem o objetivo de apresentar recursos extraclasse para complementar a formação dos estudantes, incentivando-os a explorar o tema da aula de forma autônoma e aprofundar conhecimentos.

Tempo previsto: 1 minuto.

Educação Profissional Paulista

Técnico em
Ciência de Dados

GOVERNO DO ESTADO
DE SÃO PAULO

{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_ÁREA DO PROFESSOR_**\n",
    "> \n",
    "> **Arquivo:** DADOSANO1C2B2S8A4_PROFESSOR.ipynb\n",
    "> \n",
    "> Este notebook contém a versão do professor para a Aula 4 da Semana 8. Ele inclui todo o conteúdo do aluno, além de orientações, dicas de condução e a solução comentada para cada exercício, formatadas em caixas de citação como esta.\n",
    "> \n",
    "> **Objetivo Geral da Aula Prática:** Consolidar o conhecimento da semana, levando os alunos a analisar problemas e escolher a estrutura de repetição (`for` ou `while`) e os comandos de controle (`break`, `continue`) mais adequados para cada cenário."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">TÉCNICO EM CIÊNCIA DE DADOS</h1>\n",
    "<h1 style=\"text-align: center;\">Roteiro de Atividade Prática - PROFESSOR</h1>\n",
    "
\n",
    "
\n",
    "\n",
    "**Componente:** Programação Aplicada à Ciência de Dados\n",
    "
\n",
    "**Unidade Curricular:** Lógica de Programação e Fundamentos de Python\n",
    "
\n",
    "**Tema da Semana:** Revisão geral e prática de condicionais\n",
    "<br>"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# Aula 4 - Exercícios com for e while em diferentes contextos\n",
    "\n",
    "Nesta aula prática final, vamos juntar todo o conhecimento da semana para resolver desafios. O objetivo principal é analisar cada problema e decidir qual é a melhor ferramenta (`if`, `for`, `while`, `break`) para construir a solução mais eficiente."
   ]
  },
  {

"cell_type": "markdown",
"metadata": {},
"source": [
"## Colocando em Prática"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"### Parte 1: Revisando Nossas Ferramentas\n",
"\n",
"Antes dos desafios, vamos fazer uma rápida revisão das ferramentas que temos em nosso \"cinto de utilidades\" de programação.\n",
"\n",
"#### A Ferramenta de Decisão: `if / elif / else`\n",
"É a nossa ferramenta para tomar decisões. O programa verifica uma condição e escolhe **um único caminho** para seguir. Nunca executa dois blocos ao mesmo tempo.\n",
"\n",
"#### A Ferramenta de Repetição Contada: `for`\n",
"É ideal quando sabemos **exatamente quantas vezes** queremos repetir uma ação. Usamos com `range()` para percorrer uma sequência numérica definida.\n",
"\n",
"#### A Ferramenta de Repetição Condicional: `while`\n",
"É perfeita para quando a repetição depende de uma **condição que pode mudar**. O laço continua **enquanto** a condição for verdadeira, sem sabermos de antemão quantas vezes isso levará.\n",
"\n",
"---\n",
"### Como Escolher a Ferramenta Certa?\n",
"\n",
"Ao ler um problema, faça estas perguntas:\n",
"\n",
"1. **Preciso tomar uma decisão uma única vez?**\n",
" - Sim? Use **`if / elif / else`**.\n",
"\n",
"2. **Preciso repetir algo um número fixo de vezes (ex: 10 vezes, ou para cada item de uma lista)?**\n",
" - Sim? Use **`for`**.\n",
"\n",
"3. **Preciso repetir algo até que uma condição mude (ex: até o usuário digitar 'sair', ou até a senha estar correta)?**\n",
" - Sim? Use **`while`**."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"## Parte 2: Desafios Práticos\n",
"\n",

"Analise cada problema, pense em qual ferramenta é a mais adequada e construa a solução."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"---\n",
"### Exercício 1: O Professor de Matemática Digital\n",
"\n",
"**Contexto:** Imagine que você quer criar uma ferramenta para ajudar um colega a estudar a tabuada. O programa deve ser interativo, pedindo um número e mostrando a tabuada completa dele, do 1 ao 10.\n",
"\n",
"**Exemplo:** Se o usuário digitar o número `7`, a saída do programa deve ser:\n",
"```\n",
"7 x 1 = 7\n",
"7 x 2 = 14\n",
"...\n",
"7 x 10 = 70\n",
"```"
]
},
{
"cell_type": "code",
"execution_count": 1,
"metadata": {},
"outputs": [],
"source": [
"# Escreva sua solução aqui\n",
"\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"> **_Para o Professor - Exercício 1:_**\n",
"> \n",
"> **Objetivo da Atividade:** Solidificar o uso do `for` com `range()` para um problema de repetição contada e definida.\n",
"> \n",
"> **Ponto de Atenção:** O enunciado deixa claro que a repetição vai de 1 a 10. Verifique se o aluno usa o `range(1, 11)` corretamente. Este é um bom momento para reforçar o `input()` com conversão para `int()` e a formatação do `print` com f-strings.\n",
"> \n",
"> ---\n",
"> \n",
"> **_Solução Comentada:_**"
]
},

```
  {
   "cell_type": "code",
   "execution_count": 2,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Digite um número para ver sua tabuada:  8\n"
     ]
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "\n",
      "--- Tabuada do 8 ---\n",
      "8 x 1 = 8\n",
      "8 x 2 = 16\n",
      "8 x 3 = 24\n",
      "8 x 4 = 32\n",
      "8 x 5 = 40\n",
      "8 x 6 = 48\n",
      "8 x 7 = 56\n",
      "8 x 8 = 64\n",
      "8 x 9 = 72\n",
      "8 x 10 = 80\n"
     ]
    }
   ],
   "source": [
    "# Pede um número ao usuário e converte para inteiro\n",
    "numero_tabuada = int(input(\"Digite um número para ver sua tabuada:
\"))\n",
    "\n",
    "print(f\"\\n--- Tabuada do {numero_tabuada} ---\")\n",
    "\n",
    "# O laço for é ideal, pois sabemos que a repetição acontecerá 10
vezes.\n",
    "# Usamos range(1, 11) para gerar os números de 1 a 10.\n",
    "for i in range(1, 11):\n",
    "    resultado = numero_tabuada * i\n",
    "    print(f\"{numero_tabuada} x {i} = {resultado}\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "---\n",
    "### Exercício 2: O Guardião do Enigma\n",
    "\n",
```

"**Contexto:** Você está em um jogo e encontra um guardião que bloqueia uma passagem. Ele só o deixará passar se você acertar o enigma: \"Qual a capital da França?\". O programa deve continuar fazendo a pergunta, sem parar, até que o usuário digite a resposta correta.\n",
"\n",
"**Dica:** A resposta correta é \"Paris\". Cuidado com letras maiúsculas e minúsculas! Você pode usar `.lower()` ou `.capitalize()` na resposta do usuário para padronizá-la."
]
},
{
"cell_type": "code",
"execution_count": 3,
"metadata": {},
"outputs": [],
"source": [
"# Escreva sua solução aqui\n",
"\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"> **_Para o Professor - Exercício 2:_**\n",
"> \n",
"> **Objetivo da Atividade:** Praticar o `while` para uma repetição condicional que depende de uma entrada de texto do usuário.\n",
"> \n",
"> **Dica de Condução:** Este é um ótimo momento para revisar métodos de string como `.lower()`, `.upper()` ou `.capitalize()`. Mostre aos alunos como o uso desses métodos torna a condição de parada mais robusta, aceitando \"paris\", \"Paris\" ou \"PARIS\" como respostas corretas e evitando frustração.\n",
"> \n",
"> ---\n",
"> \n",
"> **_Solução Comentada:_**"
]
},
{
"cell_type": "code",
"execution_count": 4,
"metadata": {},
"outputs": [
{
"name": "stdout",
"output_type": "stream",
"text": [
"Qual a capital da França? TESTE\n",
"Qual a capital da França? MADRI\n",
"Qual a capital da França? PARISS\n",
"Qual a capital da França? PARIS\n"
]

```
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Resposta correta! Você pode passar.\n"
     ]
    }
   ],
   "source": [
    "resposta_correta = \"Paris\"\n",
    "\n",
    "# A repetição é indefinida, então o while é a ferramenta ideal.\n",
    "resposta_usuario = \"\"\n",
    "\n",
    "while resposta_usuario.capitalize() != resposta_correta:\n",
    "    resposta_usuario = input(\"Qual a capital da França? \")\n",
    "\n",
    "print(\"Resposta correta! Você pode passar.\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "---\n",
    "### Exercício 3: O Analista de Dados do Estoque\n",
    "\n",
    "**Contexto:** Você trabalha em um armazém e recebe uma tarefa
simples de análise de dados. Você tem uma lista de códigos de produtos,
que são todos os números de 1 a 100. Seu chefe pede um relatório com la
**soma de todos os códigos que são pares**.\n",
    "\n",
    "**Sua Missão:** Crie um programa que percorra os números de 1 a 100,
verifique se cada número é par e, se for, o adicione a uma soma total. No
final, imprima o resultado."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 5,
   "metadata": {},
   "outputs": [],
   "source": [
    "# Escreva sua solução aqui\n",
    "\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Para o Professor - Exercício 3:_**\n",
    "> \n",
```

```
    "> **Objetivo da Atividade:** Combinar a repetição contada (`for`)
com uma decisão interna (`if`) e a lógica de um acumulador.\n",
    "> \n",
    "> **Ponto de Atenção:** O erro mais comum é colocar o `print` do
resultado final dentro do laço, o que mostraria a soma parcial a cada
passo. Reforce que o acumulador é inicializado fora do laço, atualizado
dentro, e o resultado final é exibido apenas após o laço terminar.\n",
    "> \n",
    "> ---\n",
    "> \n",
    "> **_Solução Comentada:_**"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 6,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "A soma de todos os códigos de produto pares é: 2550\n"
     ]
    }
   ],
   "source": [
    "# Inicializa o acumulador\n",
    "soma_dos_pares = 0\n",
    "\n",
    "# O for é ideal, pois vamos percorrer uma sequência de 1 a 100.\n",
    "for numero in range(1, 101):\n",
    "    # A cada passo, usamos um if para tomar uma decisão\n",
    "    if numero % 2 == 0:\n",
    "        # Se o número for par, o adicionamos à soma\n",
    "        soma_dos_pares = soma_dos_pares + numero\n",
    "\n",
    "# O print final é feito fora do laço\n",
    "print(f\"A soma de todos os códigos de produto pares é:
{soma_dos_pares}\")"
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "base",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
```

```
    "file_extension": ".py",
    "mimetype": "text/x-python",
    "name": "python",
    "nbconvert_exporter": "python",
    "pygments_lexer": "ipython3",
    "version": "3.12.4"
   }
  },
  "nbformat": 4,
  "nbformat_minor": 4
}
```

Educação Profissional Paulista

Técnico em **Ciência de Dados**

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de python

Laços de repetição: for

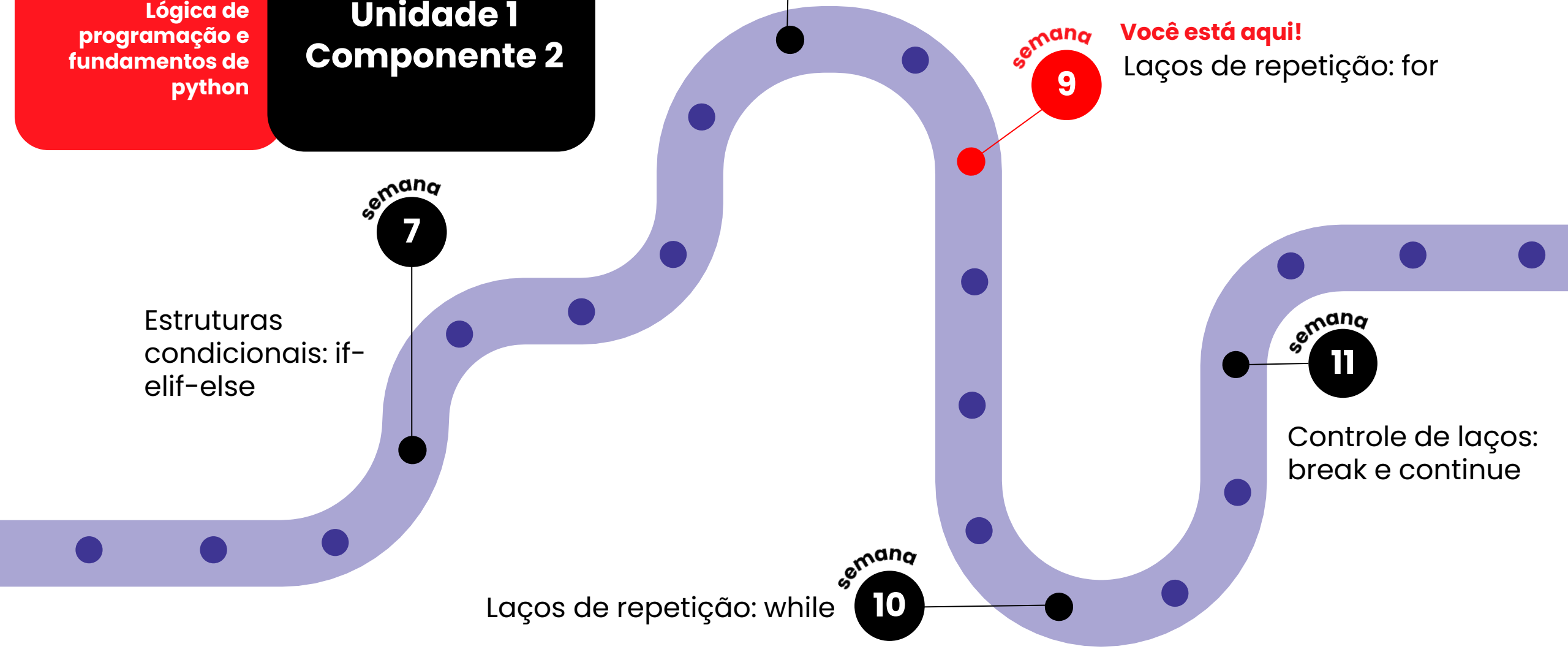
Aula 1: Laços combinados

Código da aula: DADOSANO1C2B2S9A1

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2



Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2

Você está aqui!

Laços de repetição: for

9

Aula 1: Laços combinados

Código da aula: DADOSANO1C2B2S9A1

Objetivos da aula

- Integrar estruturas de repetição e decisão em um mesmo algoritmo.

Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e à internet.
- Ferramentas de codificação para python.

Duração da aula

50 minutos.

Habilidades técnicas

- Integrar laços de repetição e estruturas condicionais para resolver problemas complexos.

Habilidades socioemocionais

- Posicionar-se com equilíbrio e escutar diferentes propostas.

Ponto de **partida**

Diálogo

Olhando para muitos dados ao mesmo tempo

Você trabalha no setor de dados de uma loja on-line de produtos de tecnologia.

Todos os dias, chegam planilhas com centenas de vendas, contendo o nome do produto, a quantidade e o valor total.

© Getty Images

O gerente quer descobrir rapidamente quais produtos vendem mais e quanto foi vendido em cada categoria, mas olhar linha por linha levaria muito tempo.

- Diante de uma lista enorme de dados, como você poderia analisar e organizar essas informações de forma rápida, sem precisar verificar uma por uma?

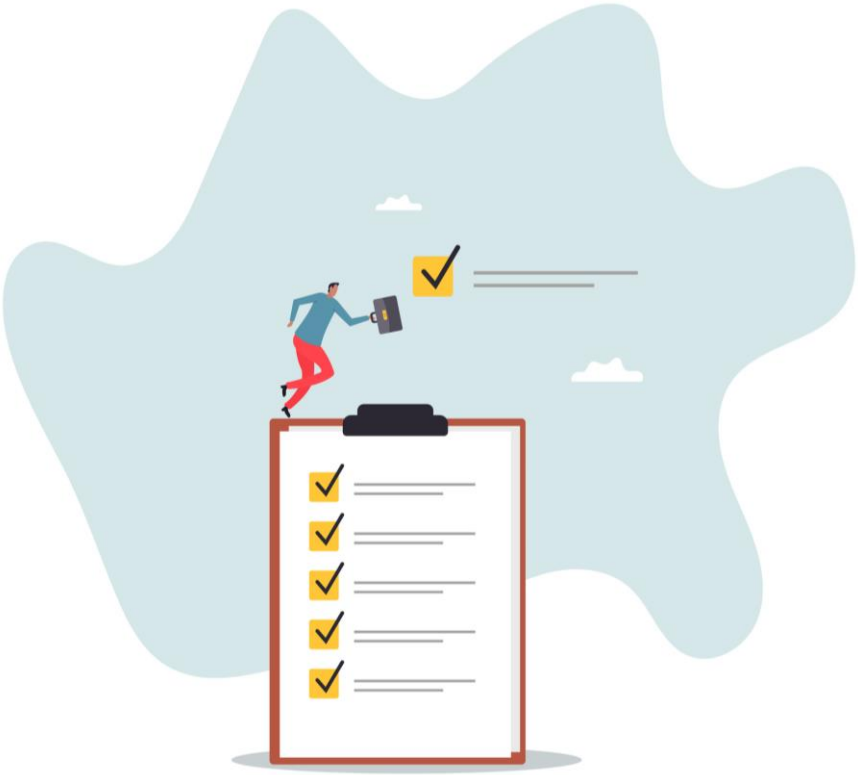
Situação fictícia produzida pela SEDUC-SP

Construindo o **conceito**

Repetir com lógica

Quando trabalhamos com dados, às vezes precisamos fazer a mesma coisa várias vezes: ler cada venda, somar valores ou verificar condições. Fazer isso manualmente seria muito demorado.

Os laços de repetição servem para isso — eles fazem o computador **repetir uma ação automaticamente**, linha por linha, até terminar o conjunto de dados.



© Getty Images

Construindo o **conceito**

A estrutura do for

O comando for é usado em python para percorrer listas e coleções de dados. Ele segue este formato:

for *variável* ***in*** *sequência*:
    instrução

Como se o **for** dissesse ao computador: “pegue uma variável por vez dessa lista e execute a instrução. Quando todos os itens forem lidos, o laço termina sozinho”.

DESTAQUE

É como se o computador estivesse “passando os olhos” sobre cada dado e decidindo o que fazer com ele, com base na instrução.

Construindo o **conceito**

Percorrendo uma lista de valores

- Antes de filtrar ou somar, é importante entender como o for percorre cada item de uma lista. Veja este exemplo simples com os valores das vendas do dia:

```
vendas = [850, 1200, 980, 1500, 600, 2100]

for valor in vendas:
    print(f"Valor da venda: R${valor}")
```

Entenda o que está acontecendo:

1. A lista vendas guarda todos os valores das transações do dia.
2. O for lê cada valor da lista e mostra na tela.
3. O código repete o mesmo comando, de forma automática, até o fim da lista.

- Resultado no console:

Valor da venda: R$ 850
Valor da venda: R$ 1 200
Valor da venda: R$ 980
Valor da venda: R$ 1 500
Valor da venda: R$ 600
Valor da venda: R$ 2 100

FICA A DICA

Esse é o primeiro passo para trabalhar com dados: percorrer a lista inteira e entender o que há dentro dela.

Construindo o **conceito**

Somando valores automaticamente

- Depois de percorrer a lista, podemos **somar os valores** para saber o total de vendas do dia.

```
vendas = [850, 1200, 980, 1500, 600, 2100]
total = 0

for valor in vendas:
    total += valor

print(f"Total vendido no dia: R${total}")
```

Entenda o que está acontecendo:

1. A variável total começa com o valor 0.
2. O for passa por cada valor da lista.
3. A cada volta, o valor é somado em total.
4. No fim, o programa mostra o total geral das vendas.

- Resultado no console:
  Total vendido no dia: R$ 7 230

- O computador faz a soma de forma automática — um passo essencial para quem trabalha com dados.

Construindo o **conceito**

Filtrando resultados

- Também podemos filtrar informações da lista.
O gerente quer saber apenas das vendas acima de R$ 1 000,00.
Veja como resolver:

```
vendas = [850, 1 200, 980, 1 500, 600, 2 100]

for valor in vendas:
    if valor > 1000:
        print(f"Venda acima do limite: R${valor}")
```

Entenda o que está acontecendo:

1. O for lê cada valor da lista.
2. O if verifica se o valor é maior que 1000.
3. Se for, o programa mostra esse valor na tela.
4. Assim, o código filtra automaticamente os valores relevantes para a análise.

- Resultado no console:

Venda acima do limite: R$ 1 200
Venda acima do limite: R$ 1 500
Venda acima do limite: R$ 2 100

DESTAQUE

Essa combinação de repetição e condição é a base da análise de dados automatizada.

Registro

Pause e
responda

O que o comando for faz em um programa?

- Armazena dados.
- Compara valores.
- Repete ações.
- Calcula resultados.

Registro

Pause e **responda**

O que o comando for faz em um programa?

- ✗ Armazena dados.
- ✗ Compara valores.
- ✓ Repete ações.
- ✗ Calcula resultados.

Ser
sempre +

Situação

Você faz parte da equipe de dados de uma empresa que está desenvolvendo um código para gerar relatórios automáticos de vendas.

Durante a reunião, cada analista apresentou uma forma diferente de resolver o mesmo problema — alguns preferem usar estruturas simples com for e if, outros sugerem funções mais avançadas. Porém, você percebe que o clima está ficando tenso, pois uma dessas pessoas aumentou o tom de voz em busca de convencer os demais.

O gestor pediu que o grupo entre em acordo sobre qual solução será usada no projeto final.

Situação fictícia produzida pela SEDUC-SP

Ser
sempre +

Ação

VIREM E CONVERSEM

- Em duplas, elaborem um pequeno roteiro de fala (de até 10 linhas) que demonstre como você se posicionaria nesse momento da reunião. Justifique.

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Compreendemos que o laço for permite ao computador repetir ações automaticamente para cada dado de uma lista.

2. Observamos como usar o for para percorrer, somar e organizar valores sem precisar fazer isso manualmente.

3. Aplicamos o for junto com o if para filtrar informações e gerar resultados de forma mais rápida e eficiente.

Saiba mais

Entenda mais sobre os laços com for! Clique no link disponível abaixo e acompanhe o vídeo.

HASHTAG PROGRAMAÇÃO. **For no Python – Criando loop com for no Python – Em menos de 5 min!** YouTube, 29 out. 2022. Disponível em: https://www.youtube.com/watch?v=d3q--ybmb9o. Acesso em: 5 nov. 2025.

Referências da aula

DOWNEY, A. B. **Pense em Python: pense como um cientista da computação**. São Paulo: Novatec, 2016.

MATTHES, E. **Curso intensivo de Python: uma introdução prática e baseada em projetos à programação**. São Paulo: Novatec, 2023.

MENEZES, N. N. C. **Introdução à programação com Python: algoritmos e lógica de programação para iniciantes**. São Paulo: Novatec, 2019.

Identidade visual: imagens © Getty Images

Orientações ao Professor

Slide 5

Orientações: professor, indicamos a seguir o tempo previsto para o desenvolvimento das seções desta aula.
Ponto de partida: 8 minutos.
Construindo o conceito: 20 minutos.
Pause e responda: 2 minutos.
Ser sempre +: 17 minutos.
O que nós aprendemos hoje: 2 minutos.
Saiba mais: 1 minuto.

Slide 6

Orientações: a seção **Ponto de partida** tem o objetivos de ativar os conhecimentos prévios dos estudantes sobre o tema da aula e estimular o pensamento crítico e as habilidades comunicativas. Por meio de uma situação-problema ou exemplo próximo da realidade do estudante, pretende-se sair da abstração conceitual e promover um diálogo dinâmico para explorar hipóteses e soluções. Espera-se também que os estudantes compartilhem eventuais experiências que possam ter com os tópicos a serem abordados em aula.

Tempo previsto: 8 minutos.

Gestão de sala de aula: assegure-se de que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos para garantir a participação de todos. Mantenha um ambiente de respeito, onde todas as opiniões são valorizadas, garantindo que todos se sintam confortáveis para expressar seus pontos de vista. Conclua a atividade resumindo as principais ideias discutidas e vinculando-as aos objetivos de aprendizagem da aula.

Condução da dinâmica: leia o slide com os estudantes.
Discussão da pergunta (5 minutos): Apresente a questão e dê tempo para os alunos pensarem e discutirem.
Apresentação (3 minutos): Peça para alguns alunos compartilharem suas ideias com a turma.

Expectativa de respostas: espera-se que os alunos mencionem ideias como:
criar um código que leia todos os dados automaticamente;
usar um programa para percorrer cada linha e fazer as contas;
fazer o computador repetir as etapas que eu faria manualmente;
montar uma lógica para o programa identificar padrões sozinho.

Slides 7 ao 11

Orientações: a seção **Construindo o conceito** tem o objetivo de ajudar o estudante a sistematizar novos conceitos teóricos relacionados com a vivência profissional.

Tempo previsto: 20 minutos.

Condução da dinâmica: apresente os slides aos estudantes com as definições de conceitos e como eles são aplicados em situações práticas.

Aprofundamento: introduzir o conceito de repetição automática como resposta à limitação humana de analisar grandes volumes de dados manualmente. O professor deve convidar os alunos a lembrar de situações em que fizeram **tarefas repetitivas**, e conectar essa ideia ao cotidiano do analista de dados, que precisa processar listas extensas (como vendas, cadastros ou registros de sensores).

Apresentar a sintaxe mínima do **for**, destacando que ela segue uma lógica de **"para cada item em um conjunto, faça algo"**. Essa lógica é comum em qualquer linguagem, mas aqui o aluno deve perceber a clareza e simplicidade do Python.
Explique a diferença entre estrutura e função: o for não calcula nada por si só — ele apenas repete um comando.
Use uma metáfora simples, como: "O for é o entregador que visita cada casa (item) de uma rua (lista) e faz uma entrega (ação)."
Mostre visualmente o recuo da indentação e o papel dos dois pontos.
Reforce que cada linha dentro do for só é executada enquanto há itens na sequência.

Permitir que o aluno **veja o for em ação pela primeira vez**, sem nenhuma outra estrutura envolvida. O objetivo é **compreender o movimento da repetição**, sem distrações de lógica condicional.
Introduzir a ideia de variável acumuladora — um dos conceitos mais importantes da lógica de programação. Aqui o aluno entende que o computador pode **guardar um resultado parcial enquanto percorre os dados**, como a soma total ou uma contagem.
Mostre que total = 0 é o ponto de partida, e que cada repetição **atualiza o valor da variável**.
Escreva na lousa um exemplo humano: "somar os valores 850 + 1 200 + ..." e depois mostre como o for faz o mesmo sozinho.
Reforce a relação entre automação e produtividade: é assim que o analista de dados consegue processar milhares de registros.

Mostrar o poder da combinação for + if — o ponto em que o código começa a tomar decisões sobre os dados. O aluno percebe que pode filtrar informações, classificar resultados ou selecionar apenas o que interessa dentro de um conjunto.

Slides 12 e 13

Orientações: o objetivo da seção **Pause e responda** é mobilizar os estudantes em relação aos conhecimentos apresentados e construídos em aula. É registro.

Tempo previsto: 2 minutos.

Resposta correta: Repete ações.
Feedback: O for faz o computador **repetir automaticamente** um conjunto de comandos para cada item de uma lista, permitindo percorrer e analisar vários dados sem precisar escrever o mesmo código várias vezes.

Slides 14 ao 15

Orientações: a seção **Ser sempre +** tem o objetivo de ajudar os estudantes a desenvolver e aprimorar competências socioemocionais, focando especificamente nas situações desafiadoras que podem surgir no ambiente profissional.

Tempo previsto: 17 minutos.

Gestão de sala de aula: Introdução (4 minutos):
Apresente a situação de maneira simples e direta. Após apresentar, organize os estudantes em duplas. Sobrando tempo ao final, verifique quais duplas gostariam de compartilhar suas ideias.

Condução da dinâmica:
Expectativa de resposta – Habilidade socioemocional: "posicionar-se com equilíbrio e escutar diferentes propostas"
Objetivo da atividade:
Verificar se os(as) estudantes são capazes de:
demonstrar escuta ativa diante de divergências;
se posicionar de forma respeitosa e colaborativa;
lidar com situações de tensão no trabalho de forma equilibrada;
buscar soluções coletivas com base em critérios objetivos e técnicos.
O que se espera do roteiro de fala elaborado pelos estudantes:
Acolhimento da diversidade de opiniões:
O estudante deve reconhecer que há diferentes caminhos para resolver o problema e que isso é natural em um ambiente colaborativo.
Postura respeitosa diante da tensão:
A fala deve evitar o confronto direto com a pessoa exaltada, propondo uma condução mais serena do debate.
Sugestão de mediação e foco na solução:
O estudante deve propor critérios objetivos ou estratégias colaborativas para comparar as propostas (ex.: testes, análise de desempenho, clareza do código).
Postura empática e colaborativa:
A fala deve reforçar o espírito de equipe e a busca pelo melhor resultado coletivo, e não pela vitória de uma única ideia.
Justificativa coerente:
A justificativa deve mostrar que a escolha do tom e do conteúdo da fala está alinhada com a manutenção de um bom ambiente de trabalho e com a qualidade do projeto.

Exemplo de resposta esperada:
“Gente, acho que todas as ideias têm pontos positivos e que a tensão só mostra o quanto todo mundo está envolvido em fazer o melhor. Talvez a gente possa escolher alguns critérios para comparar as opções, como desempenho, clareza e manutenção. Se fizermos testes curtos com as sugestões, podemos decidir com base em dados e evitar que a decisão dependa só da opinião. O mais importante é que a gente se entenda e entregue um bom resultado juntos.”
Justificativa:
Essa fala demonstra equilíbrio emocional, respeito às opiniões divergentes e foco em critérios técnicos para a tomada de decisão.
Também ajuda a reduzir o conflito sem expor diretamente o colega que elevou o tom, favorecendo um ambiente mais colaborativo.

Slide 16

Orientações: a seção **O que nós aprendemos hoje?** tem o objetivo de reforçar e esclarecer os principais conceitos discutidos, incentivando os estudantes a recordarem e articularem os principais pontos da aula.

Tempo previsto: 2 minutos.

Gestão da sala de aula:
mantenha um ambiente de diálogo aberto e respeitoso;
seja direto e objetivo nas explicações para manter a atividade dentro do tempo estipulado;
engaje os estudantes rapidamente, pedindo confirmações ou reações breves às definições apresentadas.

Condução da dinâmica:
explique aos estudantes que esse é um momento de reflexão e esclarecimento sobre os conceitos abordados na aula;
informe que será feita uma rápida revisão para assegurar o entendimento correto das definições e dos conceitos;
apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando a explicação com as informações que achar necessárias;
finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.

O que é esperado:
Os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.

Slide 17

Orientações: a seção **Saiba mais** apresenta recursos extraclasse para que os estudantes complementem sua formação. Incentive-os a continuarem explorando o tema da aula.

Tempo previsto: 1 minuto.

Educação Profissional Paulista

Técnico em **Ciência de Dados**

GOVERNO DO ESTADO
DE SÃO PAULO

Educação Profissional Paulista

Técnico em **Ciência de Dados**

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de python

Laços de repetição: for

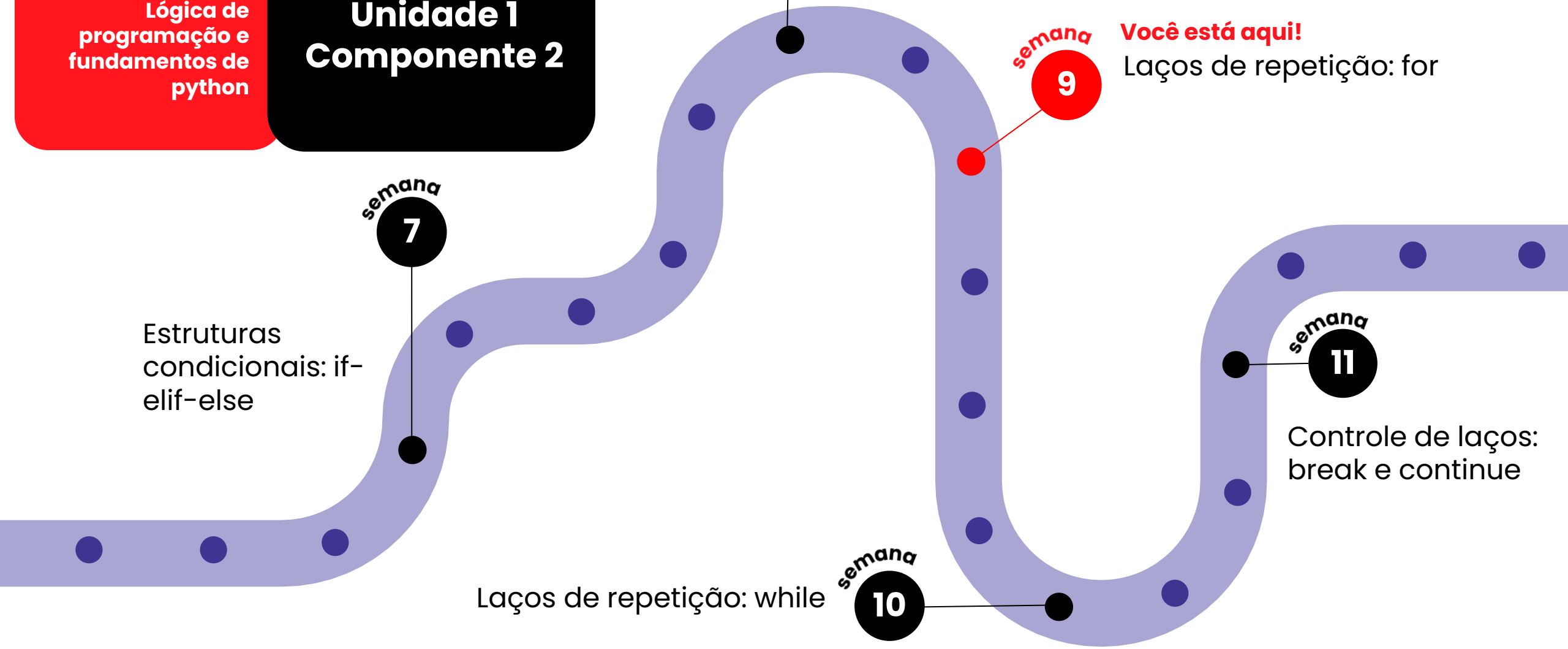
Aula 2: Laços aninhados

Código da aula: DADOSANO1C2B2S9A2

GOVERNO DO ESTADO DE SÃO PAULO

Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2



Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2

Você está aqui!

Laços de repetição: for

9

Aula 2: Laços aninhados

Código da aula: DADOSANO1C2B2S9A2

Objetivos da aula

- Construir laços aninhados para resolução de problemas com múltiplas etapas.

Recursos didáticos

- Roteiro de atividade prática.
- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e à internet.
- Ferramentas de codificação para python.

Duração da aula

50 minutos.

Habilidades técnicas

- Integrar laços de repetição e estruturas condicionais para resolver problemas complexos.

Habilidades socioemocionais

- Posicionar-se com equilíbrio e escutar diferentes propostas.

Diálogo

Relembre

Laços combinados

- O que o comando for faz em um programa python?
- Qual é o papel da variável acumuladora dentro de um laço for?
- Como o if pode ser usado junto com o for em um programa de análise de dados?

Colocando em **prática**

Explorando laços aninhados

Duplas

37 minutos

Materiais necessários

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática;
- Software Jupyter Notebook ou Google Colab.

Passo a passo

1. Baixe o arquivo “DADOSANO1C2B2S9A2AP_ALUNO” nele já contém o roteiro necessário.
2. Execute cada passo em conjunto com o professor.
3. Por fim, resolva as atividades propostas.

Baixe o roteiro dessa atividade / Baixe o material de apoio dessa atividade

Registro

Pause e **responda**

No for aninhado, qual laço percorre todos os itens a cada passo do outro?

- Laço externo.
- Laço interno.
- Nenhum.
- Ambos iguais.

Registro

Pause e **responda**

No for aninhado, qual laço percorre todos os itens a cada passo do outro?

- ✗ Laço externo.
- ✓ Laço interno.
- ✗ Nenhum.
- ✗ Ambos iguais.

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Compreendemos como os laços aninhados permitem percorrer duas listas ao mesmo tempo, criando combinações entre diferentes conjuntos de dados.

2. Observamos que o laço interno executa várias vezes para cada repetição do laço externo, formando estruturas parecidas com tabelas.

3. Aplicamos essa lógica para organizar informações simples, simulando relatórios de análise de dados.

Saiba mais

Para entender mais sobre os laços aninhados clique no link abaixo e acesse ao vídeo.

BRAZILIAN DEV. **13 – Loops aninhados – #Python essencial**. YouTube, 12 out. 2022. Disponível em: https://www.youtube.com/watch?v=P13NZ7Cu85g. Acesso em: 5 nov. 2025.

Referências da aula

DOWNEY, A. B. **Pense em Python: pense como um cientista da computação**. São Paulo: Novatec, 2016.

MATTHES, E. **Curso intensivo de Python: uma introdução prática e baseada em projetos à programação**. São Paulo: Novatec, 2023.

MENEZES, N. N. C. **Introdução à programação com Python: algoritmos e lógica de programação para iniciantes**. São Paulo: Novatec, 2019.

Identidade visual: imagens © Getty Images

Orientações ao Professor

Slide 05

Orientações: professor, indicamos a seguir o tempo previsto para o desenvolvimento das seções desta aula.
Relembre: 8 minutos.
Colocando em prática: 37 minutos.
Pause e responda: 2 minutos.
O que nós aprendemos hoje: 2 minutos.
Saiba mais: 1 minuto.

Slide 06

Orientações: a seção **Relembre** tem o objetivo de retomar conhecimentos já adquiridos por meio de atividades simples/exercícios que abordem conceitos previamente trabalhados com os estudantes.

Tempo previsto: 8 minutos.

Gestão de sala de aula: assegure-se de que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos para garantir a participação de todos. Mantenha um ambiente de respeito, onde todas as opiniões são valorizadas, garantindo que todos se sintam confortáveis para expressar seus pontos de vista. Conclua a atividade resumindo as principais ideias discutidas e vinculando-as aos objetivos de aprendizagem da aula.

Condução da dinâmica: faça as perguntas do relembre para os alunos e os incentive a participar e relembrar os conceitos apresentados nas aulas anteriores.
É importante reforçar bem os conceitos aprendidos, pois os alunos vão precisar estar com eles bem fixados para as demais aulas.

Expectativa de respostas:
Resposta 1: Ele repete automaticamente um bloco de comandos para cada item de uma lista, evitando que o programador escreva várias vezes o mesmo código.

Resposta 2: Ela guarda um valor que vai sendo atualizado a cada repetição, como a soma total das vendas ou a contagem de itens analisados.

Resposta 3: O if serve para filtrar os dados dentro do laço, permitindo que o programa execute ações apenas quando determinada condição for verdadeira.

Slide 07

Orientações: o objetivo da seção **Colocando em prática** é levar os estudantes a aplicar os conhecimentos construídos durante a aula, incentivando-os a promover autonomia, senso crítico, criatividade.

Tempo previsto: 37 minutos.

Gestão de sala de aula: Explique aos alunos que o exercício não envolve o uso de software, mas sim **a construção do raciocínio estratégico** por trás de uma boa apresentação de dados.

Condução da dinâmica:
Apresentação (5 min):
Retome brevemente o que são **laços aninhados** e mostre no projetor o exemplo já trabalhado em aula.
Explique que, nesta prática, os alunos deverão **montar seus próprios códigos**, usando listas fornecidas no enunciado, para **gerar combinações e estruturas de dados simples** (como relatórios de cidade × produto ou mês × indicador).
Enfatize que o foco está em **raciocinar como os laços percorrem as listas** e **interpretar o resultado gerado**.
Execução da atividade (25 min):
Oriente os alunos a trabalharem em duplas para discutir a lógica e construir os códigos dos dois exercícios:
Exercício 1: Cruzamento de cidades e produtos.
Exercício 2: Relatório de indicadores por mês.
Peça que cada dupla copie as listas apresentadas, escreva o código completo e execute o resultado, observando o comportamento do laço interno e externo.
Circule pela sala, verificando se os alunos compreenderam a hierarquia dos laços e identificando se os prints estão formatados corretamente.
Encerramento (7 min):
Selecione **2 ou 3 duplas** para apresentar brevemente seus códigos e resultados.

Expectativas de respostas:
Estão descritas no **roteiro do professor** DADOSANO1C2B2S9A2AP_PROFESSOR.ipynb.

Slides 08 e 09

Orientações: o objetivo da seção **Pause e responda** é mobilizar os estudantes em relação aos conhecimentos apresentados e construídos em aula. É registro.

Tempo previsto: 2 minutos.

Resposta certa: Laço interno.
Feedback: Em laços aninhados, para cada iteração do laço externo, o laço interno executa todas as suas repetições, permitindo cruzar duas dimensões de dados.

Slide 10

Orientações: a seção **O que nós aprendemos hoje?** tem o objetivo de reforçar e esclarecer os principais conceitos discutidos, incentivando os estudantes a recordarem e articularem os principais pontos da aula.

Tempo previsto: 2 minutos.

Gestão da sala de aula:
mantenha um ambiente de diálogo aberto e respeitoso;
seja direto e objetivo nas explicações para manter a atividade dentro do tempo estipulado;
engaje os estudantes rapidamente, pedindo confirmações ou reações breves às definições apresentadas.

Condução da dinâmica:
explique aos estudantes que esse é um momento de reflexão e esclarecimento sobre os conceitos abordados na aula;
informe que será feita uma rápida revisão para assegurar o entendimento correto das definições e dos conceitos;
apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando a explicação com as informações que achar necessárias;
finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.

O que é esperado:
Os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.

Slide 11

Orientações: a seção **Saiba mais** apresenta recursos extraclasse para que os estudantes complementem sua formação. Incentive-os a continuarem explorando o tema da aula.

Tempo previsto: 1 minuto.

Educação Profissional Paulista

Técnico em **Ciência de Dados**

GOVERNO DO ESTADO
DE SÃO PAULO

{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">TÉCNICO EM CIÊNCIA DE DADOS</h1>\n",
    "<h1 style=\"text-align: center;\">Roteiro de Atividade Prática</h1>\n",
    "
\n",
    "
\n",
    "\n",
    "**Componente -** Programação Aplicada à Ciência de Dados\n",
    "
\n",
    "**Unidade Curricular -** Lógica de Programação e Fundamentos de Python\n",
    "
\n",
    "**Tema da Semana -** Laços de repetição: for\n",
    "
\n",
    "**Semana 9**\n",
    "
\n",
    "**Aula 2 -** Laços aninhados"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">Explorando laços aninhados</h1> "
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Objetivos da atividade\n",
    "\n",
    "* Entender o que é um laço de repetição dentro de outro (for dentro de for).\n",
    "\n",
    "* Observar como os laços aninhados permitem cruzar informações entre listas.\n",
    "\n",
    "* Desenvolver pequenos códigos em Python que gerem combinações ou padrões simples de dados.\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Base teórica\n",
    "\n",

"Um **laço aninhado** é uma estrutura de repetição onde um **laço está dentro de outro.**\n",
"\n",
"Ele é usado quando precisamos **percorrer duas ou mais dimensões de dados** — por exemplo, uma lista de listas, uma tabela ou o cruzamento de duas coleções de informações.\n",
"\n",
"#### A Lógica\n",
"\n",
"* O laço externo controla o ciclo principal (como as linhas de uma tabela);\n",
"\n",
"* O laço interno executa várias vezes para cada repetição do laço externo (como as colunas de cada linha).\n",
"\n",
"#### A sintaxe\n",
"\n",
"```python\n",
"for item_externo in conjunto_externo:\n",
"   for item_interno in conjunto_interno:\n",
"   # código a ser executado em cada combinação\n",
"```\n",
"\n",
"### Raciocínio - Um passo de cada vez:\n",
"\n",
"1. O python entra no primeiro laço (for item_externo) e escolhe um elemento.\n",
"\n",
"2. Para cada elemento externo, o segundo laço (for item_interno) percorre todos os elementos internos.\n",
"\n",
"3. O bloco de código interno é executado várias vezes, até que o laço interno termine.\n",
"\n",
"4. Em seguida, o Python retorna ao laço externo e repete o processo para o próximo elemento.\n",
"\n",
"#### Imagine um relógio:\n",
"\n",
"* O **ponteiro das horas** (laço externo) avança devagar;\n",
"\n",
"* O **ponteiro dos minutos** (laço interno) completa várias voltas antes que a hora mude.\n",
"\n",
"Assim também acontece com os laços aninhados — para cada passo do laço de fora, o laço de dentro percorre todo o seu ciclo."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"#### **Exemplo — Cruzando cidades e produtos**"
]

```
  },
  {
   "cell_type": "code",
   "execution_count": 1,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Cidade: SP | Produto: Notebook\n",
      "Cidade: SP | Produto: Mouse\n",
      "Cidade: RJ | Produto: Notebook\n",
      "Cidade: RJ | Produto: Mouse\n"
     ]
    }
   ],
   "source": [
    "# Lista de cidades analisadas\n",
    "cidades = [\"SP\", \"RJ\"]\n",
    "\n",
    "# Lista de produtos vendidos\n",
    "produtos = [\"Notebook\", \"Mouse\"]\n",
    "\n",
    "# Para cada cidade (laço externo)\n",
    "for cidade in cidades:\n",
    "    # Para cada produto (laço interno)\n",
    "    for produto in produtos:\n",
    "        # Exibe a combinação cidade-produto\n",
    "        print(\"Cidade:\", cidade, \"| Produto:\", produto)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Raciocínio por trás do código_**\n",
    "> \n",
    "> * O primeiro laço (externo) percorre todas as cidades da
lista.\n",
    ">\n",
    "> * Para cada cidade, o segundo laço (interno) percorre todos os
produtos.\n",
    ">\n",
    "> * A cada repetição, o Python cria uma combinação de cidade e
produto."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Construindo padrões com laços aninhados\n",
    "\n",
```

"Os laços aninhados também podem ser usados para criar padrões visuais simples no console.\n",
"Esse tipo de exercício ajuda a entender como os laços se repetem e como controlar linhas e colunas de forma organizada.\n",
"\n",
"A ideia é pensar como se estivéssemos desenhando uma pequena tabela:\n",
"\n",
"* O laço externo representa as linhas;\n",
"\n",
"* O laço interno representa as colunas.\n",
"\n",
"Esse tipo de estrutura é comum quando queremos exibir informações de forma alinhada, simulando uma planilha simples."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"#### **Exemplo: organizando vendas por cidade e produto**\n",
"\n",
"Um analista quer visualizar quantos produtos diferentes são oferecidos em cada cidade.\n",
"\n",
"Ele ainda não tem uma planilha pronta, então vai gerar uma estrutura de tabela no console usando laços aninhados."
]
},
{
"cell_type": "code",
"execution_count": 2,
"metadata": {},
"outputs": [
{
"name": "stdout",
"output_type": "stream",
"text": [
"CIDADE | PRODUTOS DISPONÍVEIS\n",
"------------------------------\n",
"SP - Notebook\n",
"SP - Mouse\n",
"SP - Teclado\n",
"\n",
"RJ - Notebook\n",
"RJ - Mouse\n",
"RJ - Teclado\n",
"\n",
"MG - Notebook\n",
"MG - Mouse\n",
"MG - Teclado\n",
"\n"
]
}

```
   ],
   "source": [
    "# Listas com as cidades e os produtos disponíveis\n",
    "cidades = [\"SP\", \"RJ\", \"MG\"]\n",
    "produtos = [\"Notebook\", \"Mouse\", \"Teclado\"]\n",
    "\n",
    "# Cabeçalho da tabela\n",
    "print(\"CIDADE | PRODUTOS DISPONÍVEIS\")\n",
    "print(\"------------------------------\")\n",
    "\n",
    "# O laço externo percorre as cidades (linhas)\n",
    "for cidade in cidades:\n",
    "    # O laço interno percorre os produtos (colunas)\n",
    "    for produto in produtos:\n",
    "        print(cidade, \"-\", produto)\n",
    "    print()  # pula uma linha entre as cidades\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Raciocínio:_**\n",
    "> \n",
    "> O laço externo (for cidade in cidades) percorre as cidades, linha
por linha.\n",
    ">\n",
    "> O laço interno (for produto in produtos) percorre os produtos e os
associa à cidade atual.\n",
    ">\n",
    "> O resultado é uma visualização estruturada de dados, como se fosse
uma mini-tabela."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "---\n",
    "### **Colocando em Prática**"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### **Exercício 1- Somando o total de vendas do dia**\n",
    "\n",
    "A equipe de dados da loja TechVendas precisa calcular o total de
vendas do dia a partir da lista abaixo, que mostra o valor de cada pedido
realizado.\n",
    "\n",
    "O relatório será usado para comparar o resultado com a meta
diária.\n",
```

"\n",
"Os valores estão armazenados nesta lista:\n",
"\n",
"vendas = [250, 320, 180, 400, 150, 280, 310, 200]\n",
"\n",
"#### Tarefa\n",
"\n",
"1. Crie uma variável chamada total e inicialize com zero.\n",
"\n",
"2. Use um laço for para percorrer todos os valores da lista.\n",
"\n",
"3. A cada repetição, some o valor atual ao total.\n",
"\n",
"4. Ao final, mostre o resultado no formato:\n",
"\n",
"Total de vendas do dia: R$ <valor>\n"
]
},
{
"cell_type": "code",
"execution_count": 3,
"metadata": {},
"outputs": [],
"source": [
"# Escreva sua solução aqui\n",
"\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"---\n",
"#### **Exercício 2- Mapeando indicadores mensais**\n",
"\n",
"O setor de análise quer gerar uma tabela simples que mostre quais indicadores estão sendo avaliados em cada mês do trimestre.\n",
"\n",
"Os dados disponíveis são:\n",
"\n",
"meses = [\"Janeiro\", \"Fevereiro\", \"Março\"]\n",
"\n",
"indicadores = [\"Vendas\", \"Clientes Ativos\", \"Lucro\"]\n",
"\n",
"#### Tarefa\n",
"\n",
"Utilize laços aninhados (for dentro de for) para exibir os indicadores de cada mês, seguindo o formato:\n",
"\n",
"Janeiro - Vendas\n",
"\n",
"Janeiro - Clientes Ativos\n",
"\n",
"Janeiro - Lucro\n",

```
  "\n",
  "Fevereiro - Vendas\n",
  "\n",
  "...\n",
  "\n"
 ]
},
{
 "cell_type": "code",
 "execution_count": 4,
 "metadata": {},
 "outputs": [],
 "source": [
  "# Escreva sua solução aqui\n",
  "\n"
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "---\n",
  "#### <h1 style=\"text-align: center;\">Gabarito comentado</h1> "
 ]
},
{
 "cell_type": "markdown",
 "metadata": {},
 "source": [
  "#### Exercício 1- Somando o total de vendas do dia"
 ]
},
{
 "cell_type": "code",
 "execution_count": 5,
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "Total de vendas do dia: R$ 2090\n"
   ]
  }
 ],
 "source": [
  "# Escreva sua solução aqui\n",
  "\n",
  "# Lista com os valores das vendas realizadas no dia\n",
  "vendas = [250, 320, 180, 400, 150, 280, 310, 200]\n",
  "\n",
  "# Variável acumuladora, começa em zero\n",
  "total = 0\n",
  "\n",
```

```
    "# O laço percorre cada valor da lista 'vendas'\n",
    "for valor in vendas:\n",
    "    total += valor    # soma o valor atual ao total acumulado\n",
    "\n",
    "# Exibe o resultado final\n",
    "print(\"Total de vendas do dia: R$\", total)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Explicação para o professor:_**\n",
    "> \n",
    "> * define a lista de dados que será percorrida, representando os
valores de vendas do dia.\n",
    ">\n",
    "> * inicializa a variável total com o valor zero — ponto de partida
para o cálculo acumulado.\n",
    ">\n",
    "> * Laço for: percorre cada elemento da lista, um por vez.\n",
    ">\n",
    "> * Dentro do laço: o operador += soma o valor atual (valor) ao
acumulado (total).\n",
    ">\n",
    "> * Saída final: imprime o total com uma mensagem formatada,
simulando o relatório de vendas."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "#### Exercício 2 — Mapeando indicadores mensais"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 6,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Relatório de Indicadores por Mês\n",
      "--------------------------------\n",
      "Janeiro - Vendas\n",
      "Janeiro - Clientes Ativos\n",
      "Janeiro - Lucro\n",
      "\n",
      "Fevereiro - Vendas\n",
      "Fevereiro - Clientes Ativos\n",
      "Fevereiro - Lucro\n",
```

```
      "\n",
      "Março - Vendas\n",
      "Março - Clientes Ativos\n",
      "Março - Lucro\n",
      "\n"
     ]
    }
   ],
   "source": [
    "# Escreva sua solução aqui\n",
    "\n",
    "# Listas de dados fornecidas\n",
    "meses = [\"Janeiro\", \"Fevereiro\", \"Março\"]\n",
    "indicadores = [\"Vendas\", \"Clientes Ativos\", \"Lucro\"]\n",
    "\n",
    "# Cabeçalho para a tabela de análise\n",
    "print(\"Relatório de Indicadores por Mês\")\n",
    "print(\"-------------------------------\")\n",
    "\n",
    "# Laço externo percorre os meses\n",
    "for mes in meses:\n",
    "    # Laço interno percorre os indicadores\n",
    "    for indicador in indicadores:\n",
    "        # Exibe o par mês-indicador\n",
    "        print(f\"{mes} - {indicador}\")\n",
    "    print()  # separa visualmente cada mês\n",
    "\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Explicação para o professor:_**\n",
    "> \n",
    "> * As listas representam categorias de análise: meses e
indicadores.\n",
    ">\n",
    "> * O laço externo percorre cada mês da lista meses.\n",
    ">\n",
    "> * O laço interno percorre todos os indicadores e mostra cada
combinação.\n",
    ">\n",
    "> * O print(f\"{mes} - {indicador}\") organiza os resultados de
forma simples e clara.\n",
    "> \n",
    "> * O print() fora do segundo laço adiciona um espaço entre os
meses, facilitando a leitura."
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "base",
```

```
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.4"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 4
}
```

{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">TÉCNICO EM CIÊNCIA DE DADOS</h1>\n",
    "<h1 style=\"text-align: center;\">Roteiro de Atividade Prática</h1>\n",
    "
\n",
    "
\n",
    "\n",
    "**Componente -** Programação Aplicada à Ciência de Dados\n",
    "
\n",
    "**Unidade Curricular -** Lógica de Programação e Fundamentos de Python\n",
    "
\n",
    "**Tema da Semana -** Laços de repetição: for\n",
    "
\n",
    "**Semana 9**\n",
    "
\n",
    "**Aula 2 -** Laços aninhados"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">Explorando laços aninhados</h1> "
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Objetivos da atividade\n",
    "\n",
    "* Entender o que é um laço de repetição dentro de outro (for dentro de for).\n",
    "\n",
    "* Observar como os laços aninhados permitem cruzar informações entre listas.\n",
    "\n",
    "* Desenvolver pequenos códigos em python que gerem combinações ou padrões simples de dados.\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Base teórica\n",
    "\n",

"Um **laço aninhado** é uma estrutura de repetição onde um **laço está dentro de outro.**\n",
"\n",
"Ele é usado quando precisamos **percorrer duas ou mais dimensões de dados** — por exemplo, uma lista de listas, uma tabela ou o cruzamento de duas coleções de informações.\n",
"\n",
"#### A lógica\n",
"\n",
"* O laço externo controla o ciclo principal (como as linhas de uma tabela);\n",
"\n",
"* O laço interno executa várias vezes para cada repetição do laço externo (como as colunas de cada linha).\n",
"\n",
"#### A sintaxe\n",
"\n",
"```python\n",
"for item_externo in conjunto_externo:\n",
"   for item_interno in conjunto_interno:\n",
"   # código a ser executado em cada combinação\n",
"```\n",
"\n",
"### Raciocínio - Um passo de cada vez:\n",
"\n",
"1. O python entra no primeiro laço (for item_externo) e escolhe um elemento.\n",
"\n",
"2. Para cada elemento externo, o segundo laço (for item_interno) percorre todos os elementos internos.\n",
"\n",
"3. O bloco de código interno é executado várias vezes, até que o laço interno termine.\n",
"\n",
"4. Em seguida, o python retorna ao laço externo e repete o processo para o próximo elemento.\n",
"\n",
"#### Imagine um relógio:\n",
"\n",
"* O **ponteiro das horas** (laço externo) avança devagar;\n",
"\n",
"* O **ponteiro dos minutos** (laço interno) completa várias voltas antes que a hora mude.\n",
"\n",
"Assim também acontece com os laços aninhados — para cada passo do laço de fora, o laço de dentro percorre todo o seu ciclo."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"#### **Exemplo — Cruzando cidades e produtos**"
]

```
  },
  {
   "cell_type": "code",
   "execution_count": 1,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Cidade: SP | Produto: Notebook\n",
      "Cidade: SP | Produto: Mouse\n",
      "Cidade: RJ | Produto: Notebook\n",
      "Cidade: RJ | Produto: Mouse\n"
     ]
    }
   ],
   "source": [
    "# Lista de cidades analisadas\n",
    "cidades = [\"SP\", \"RJ\"]\n",
    "\n",
    "# Lista de produtos vendidos\n",
    "produtos = [\"Notebook\", \"Mouse\"]\n",
    "\n",
    "# Para cada cidade (laço externo)\n",
    "for cidade in cidades:\n",
    "    # Para cada produto (laço interno)\n",
    "    for produto in produtos:\n",
    "        # Exibe a combinação cidade-produto\n",
    "        print(\"Cidade:\", cidade, \"| Produto:\", produto)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Raciocínio por trás do código_**\n",
    "> \n",
    "> * O primeiro laço (externo) percorre todas as cidades da
lista.\n",
    ">\n",
    "> * Para cada cidade, o segundo laço (interno) percorre todos os
produtos.\n",
    ">\n",
    "> * A cada repetição, o Python cria uma combinação de cidade e
produto."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### Construindo padrões com laços aninhados\n",
    "\n",
```

"Os laços aninhados também podem ser usados para criar padrões visuais simples no console.\n",
"Esse tipo de exercício ajuda a entender como os laços se repetem e como controlar linhas e colunas de forma organizada.\n",
"\n",
"A ideia é pensar como se estivéssemos desenhando uma pequena tabela:\n",
"\n",
"* O laço externo representa as linhas;\n",
"\n",
"* O laço interno representa as colunas.\n",
"\n",
"Esse tipo de estrutura é comum quando queremos exibir informações de forma alinhada, simulando uma planilha simples."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"#### **Exemplo: organizando vendas por cidade e produto**\n",
"\n",
"Um analista quer visualizar quantos produtos diferentes são oferecidos em cada cidade.\n",
"\n",
"Ele ainda não tem uma planilha pronta, então vai gerar uma estrutura de tabela no console usando laços aninhados."
]
},
{
"cell_type": "code",
"execution_count": 3,
"metadata": {},
"outputs": [
{
"name": "stdout",
"output_type": "stream",
"text": [
"CIDADE | PRODUTOS DISPONÍVEIS\n",
"------------------------------\n",
"SP - Notebook\n",
"SP - Mouse\n",
"SP - Teclado\n",
"\n",
"RJ - Notebook\n",
"RJ - Mouse\n",
"RJ - Teclado\n",
"\n",
"MG - Notebook\n",
"MG - Mouse\n",
"MG - Teclado\n",
"\n"
]
}

```
   ],
   "source": [
    "# Listas com as cidades e os produtos disponíveis\n",
    "cidades = [\"SP\", \"RJ\", \"MG\"]\n",
    "produtos = [\"Notebook\", \"Mouse\", \"Teclado\"]\n",
    "\n",
    "# Cabeçalho da tabela\n",
    "print(\"CIDADE | PRODUTOS DISPONÍVEIS\")\n",
    "print(\"------------------------------\")\n",
    "\n",
    "# O laço externo percorre as cidades (linhas)\n",
    "for cidade in cidades:\n",
    "    # O laço interno percorre os produtos (colunas)\n",
    "    for produto in produtos:\n",
    "        print(cidade, \"-\", produto)\n",
    "    print()  # pula uma linha entre as cidades\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Raciocínio:_**\n",
    "> \n",
    "> O laço externo (for cidade in cidades) percorre as cidades, linha
por linha.\n",
    ">\n",
    "> O laço interno (for produto in produtos) percorre os produtos e os
associa à cidade atual.\n",
    ">\n",
    "> O resultado é uma visualização estruturada de dados, como se fosse
uma mini-tabela."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "---\n",
    "### **Colocando em Prática**"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "### **Exercício 1- Somando o total de vendas do dia**\n",
    "\n",
    "A equipe de dados da loja fictícia TechVendas precisa calcular o
total de vendas do dia a partir da lista abaixo, que mostra o valor de
cada pedido realizado.\n",
    "\n",
    "O relatório será usado para comparar o resultado com a meta
diária.\n",
```

"\n",
"Os valores estão armazenados nesta lista:\n",
"\n",
"vendas = [250, 320, 180, 400, 150, 280, 310, 200]\n",
"\n",
"#### Tarefa\n",
"\n",
"1. Crie uma variável chamada total e inicialize com zero.\n",
"\n",
"2. Use um laço for para percorrer todos os valores da lista.\n",
"\n",
"3. A cada repetição, some o valor atual ao total.\n",
"\n",
"4. Ao final, mostre o resultado no formato:\n",
"\n",
"Total de vendas do dia: R$ <valor>"
]
},
{
"cell_type": "code",
"execution_count": null,
"metadata": {},
"outputs": [],
"source": [
"# Escreva sua solução aqui\n",
"\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"---\n",
"#### **Exercício 2- Mapeando indicadores mensais**\n",
"\n",
"O setor de análise quer gerar uma tabela simples que mostre quais indicadores estão sendo avaliados em cada mês do trimestre.\n",
"\n",
"Os dados disponíveis são:\n",
"\n",
"meses = [\"Janeiro\", \"Fevereiro\", \"Março\"]\n",
"\n",
"indicadores = [\"Vendas\", \"Clientes Ativos\", \"Lucro\"]\n",
"\n",
"#### Tarefa\n",
"\n",
"Utilize laços aninhados (for dentro de for) para exibir os indicadores de cada mês, seguindo o formato:\n",
"\n",
"Janeiro - Vendas\n",
"\n",
"Janeiro - Clientes Ativos\n",
"\n",
"Janeiro - Lucro\n",

```
    "\n",
    "Fevereiro - Vendas\n",
    "\n",
    "...\n",
    "\n"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 4,
   "metadata": {},
   "outputs": [],
   "source": [
    "# Escreva sua solução aqui\n",
    "\n"
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "base",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.4"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 4
}
```

Educação Profissional Paulista

Técnico em **Ciência de Dados**

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de python

Laços de repetição: for

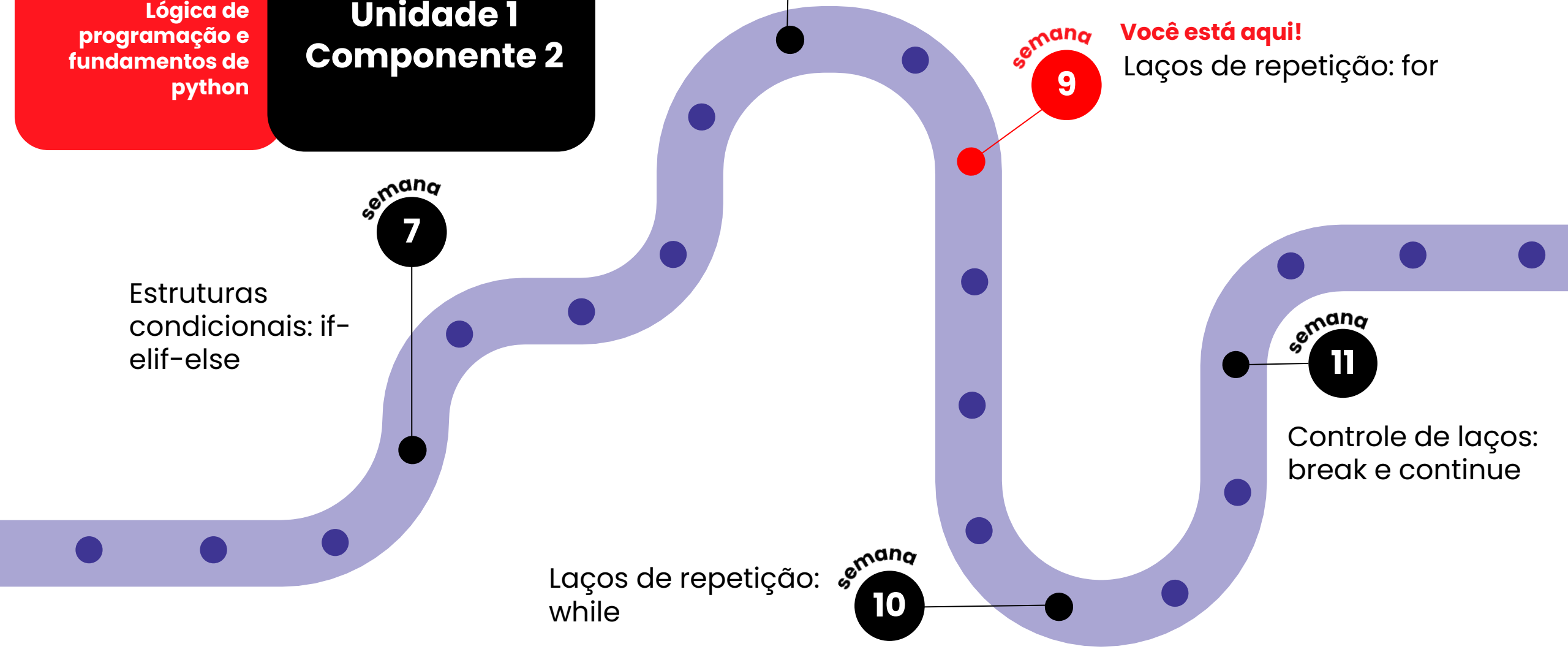
Aula 3: Aplicações práticas

Código da aula: DADOSANO1C2B2S9A3

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2



Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2

Você está aqui!

Laços de repetição: for

9

Aula 3: Aplicações práticas

Código da aula: DADOSANO1C2B2S9A3

Objetivos da aula

- Desenvolver soluções que envolvam lógica iterativa em diferentes camadas.

Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e à internet.
- Ferramentas de codificação para python.

Duração da aula

50 minutos.

Habilidades técnicas

- Integrar laços de repetição e estruturas condicionais para resolver problemas complexos.

Habilidades socioemocionais

- Posicionar-se com equilíbrio e escutar diferentes propostas.

Diálogo

Relembre

Laços aninhados

- O que significa ter um laço for dentro de outro em python?
- Qual é o papel do laço externo em uma estrutura aninhada?
- Como os laços aninhados ajudam na análise de dados?

Construindo o **conceito**

O problema dos relatórios incompletos

UM PASSO DE CADA VEZ

Você é técnico em ciência de dados em uma loja online que vende produtos em várias cidades. Todos os dias, o sistema gera uma lista com os pedidos do dia, contendo a **cidade**, o **produto** e o **valor da compra**.

No entanto, o relatório não traz:

- o total de vendas do dia;
- quantos produtos foram vendidos em cada cidade.

O gerente precisa dessas informações para planejar o estoque e avaliar o desempenho das filiais. O desafio é usar laços de repetição para resolver essas duas tarefas com o mesmo conjunto de dados.

Situação fictícia produzida pela SEDUC-SP

Continua

Construindo o **conceito**

Descobrindo o total vendido no dia

UM PASSO DE CADA VEZ

Para somar os valores de todas as vendas, usamos um laço que percorre cada registro, identificando o campo **numérico (valor da compra)**.

Como o código funciona:

- O laço percorre cada linha da lista (registro).
- registro[2] representa o valor da venda (posição do campo numérico).
- A cada repetição, o valor é somado ao total.

- O laço atua como um “leitor automático” que soma os dados linha por linha, sem precisar abrir uma planilha.

```
# Lista de pedidos: cidade, produto e valor
pedidos = [
    ["São Paulo", "Notebook", 2500],
    ["Rio de Janeiro", "Mouse", 80],
    ["Belo Horizonte", "Teclado", 120],
    ["São Paulo", "Mouse", 90]
]

# Somar todos os valores
total = 0
for registro in pedidos:
    total += registro[2]

print("Total vendido no dia: R$", total)
```

- Resultado no console:
  Total vendido no dia: R$ 2 790

Construindo o **conceito**

Contando quantos produtos foram vendidos por cidade

UM PASSO DE CADA VEZ

Agora o gerente quer saber **quantas vendas** ocorreram em cada cidade.

- Usamos um laço aninhado para cruzar as cidades com os pedidos e contar as ocorrências.

```
cidades = ["São Paulo", "Rio de Janeiro", "Belo Horizonte"]

for cidade in cidades:              # laço externo percorre as cidades
    contagem = 0
    for pedido in pedidos:          # laço interno percorre os registros
        if pedido[0] == cidade:     # compara a cidade do pedido
            contagem += 1
    print(cidade, "teve", contagem, "venda(s).")
```

DESTAQUE

Essa é a lógica por trás das tabelas dinâmicas e dos agrupamentos de dados em ferramentas profissionais.

Como o código funciona:

- O laço externo define a cidade que está sendo analisada.
- O laço interno percorre os pedidos e soma apenas os que pertencem àquela cidade.
- O resultado é uma visão resumida por cidade, como em um relatório de agrupamento.

- Resultado no console:

São Paulo teve 2 venda(s).
Rio de Janeiro teve 1 venda(s).
Belo Horizonte teve 1 venda(s).

Construindo o **conceito**

Aplicação dos laços

Produzido pela SEDUC-SP com ©Getty Images.

Colocando em **prática**

Pensar antes de programar

Você e sua dupla trabalham no setor de dados de uma empresa de e-commerce.

O gerente quer entender como os laços de repetição podem ajudar a automatizar tarefas simples, como calcular totais e contar registros, mas ele **não** entende de programação.

Situação fictícia produzida pela SEDUC-SP

Em duplas, pensem e escrevam em frases curtas o raciocínio lógico necessário para:

- Somar o total de vendas do dia;
- Contar quantas vendas foram realizadas em cada cidade.

Não programem ainda. Conversem e registrem como o computador deve pensar — em que ordem as ações acontecem, o que deve ser percorrido, somado ou comparado.

17 minutos

Duplas

Pause e
responda

O que o laço for permite fazer ao analisar uma lista de registros com textos e números?

- Alterar o tipo dos dados.
- Excluir itens da lista.
- Organizar os dados em ordem.
- Comparar e calcular valores.

Pause e **responda**

O que o laço for permite fazer ao analisar uma lista de registros com textos e números?

- ✗ Alterar o tipo dos dados.
- ✗ Excluir itens da lista.
- ✗ Organizar os dados em ordem.
- ✓ Comparar e calcular valores.

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Compreendemos como os laços de repetição podem ser usados para resolver problemas reais de análise de dados.

2. Observamos que o for permite percorrer informações textuais e numéricas, aplicando regras e cálculos de forma automatizada.

3. Aplicamos o raciocínio lógico para somar, filtrar e cruzar dados, tornando as análises mais rápidas e precisas.

Saiba mais

Entenda mais sobre o por que aprender a usar o python para ciência de dados acessando o vídeo disponível abaixo no link.

FELIPE CABRERA – BYLEARN. **5 motivos para usar Python para ciência de dados?** YouTube, 9 jul. 2021. Disponível em: https://www.youtube.com/watch?v=rnm5MiuQGj0. Acesso em: 5 nov. 2025.

Referências da aula

MENEZES, N. N. C. **Introdução à programação com Python: algoritmos e lógica de programação para iniciantes**. São Paulo: Novatec, 2019.

PYTHON SOFTWARE FOUNDATION. **Documentação Python 3.14.0**, [s.d.]. Disponível em: https://docs.python.org/pt-br/3/. Acesso em: 5 nov. 2025.

RAMALHO, L. **Python fluente: programação clara, concisa e eficiente**. São Paulo: Novatec, 2023.

Identidade visual: imagens © Getty Images

Orientações ao Professor

Slide 5

Orientações: professor, indicamos a seguir o tempo previsto para o desenvolvimento das seções desta aula.
Relembre: 8 minutos.
Construindo o conceito: 20 minutos.
Colocando em prática: 17 minutos.
Pause e responda: 2 minutos.
O que nós aprendemos hoje: 2 minutos.
Saiba mais: 1 minuto.

Slides 6 e 7

Orientações: a seção **Relembre** tem o objetivo de retomar conhecimentos já adquiridos por meio de atividades simples/exercícios que abordem conceitos previamente trabalhados com os estudantes.

Tempo previsto: 8 minutos.

Gestão de sala de aula: assegure-se de que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos para garantir a participação de todos. Mantenha um ambiente de respeito, onde todas as opiniões são valorizadas, garantindo que todos se sintam confortáveis para expressar seus pontos de vista. Conclua a atividade resumindo as principais ideias discutidas e vinculando-as aos objetivos de aprendizagem da aula.

Condução da dinâmica: faça as perguntas do relembre para os alunos e os incentive a participar e relembrar os conceitos apresentados nas aulas anteriores. É importante reforçar bem os conceitos aprendidos, pois os alunos vão precisar estar com eles bem fixados para as demais aulas.

Expectativa de respostas:
Resposta 1: Significa que um bloco de repetição é executado várias vezes dentro de outro, permitindo percorrer dois conjuntos de dados ao mesmo tempo.

Resposta 2: O laço externo controla o ciclo principal, indicando quantas vezes o laço interno será executado.

Resposta 3: Eles permitem criar combinações e organizar informações de forma estruturada, simulando linhas e colunas de uma tabela de dados.

Slides 08 ao 10

Orientações: a seção **Construindo o conceito** tem o objetivo de ajudar o estudante a sistematizar novos conceitos teóricos relacionados com a vivência profissional.

Tempo previsto: 20 minutos.

Condução da dinâmica: apresente os slides aos estudantes com as definições de conceitos e como eles são aplicados em situações práticas.

Aprofundamento: explique que em uma rotina real, o cientista de dados recebe listas ou tabelas com dados brutos — sem totais, contagens ou filtros prontos. Mostre que o papel do cientista de dados é **organizar e transformar** esses dados em **informações úteis**.

Mostre que o laço percorre uma estrutura (no caso, uma lista de listas) e **extrai apenas o campo relevante**. Reforce a importância de entender **a posição dos dados** dentro de cada linha (registro[2] representa o valor numérico). Reforce que o índice em Python começa com zero, por isso o valor é o registro 2. Cidade está no índice 0, produto está no índice 1 e valor no índice 2.

Mostre que o segundo exemplo introduz a **análise cruzada** entre categorias (cidade × pedidos). O laço externo controla uma categoria, enquanto o interno percorre os dados para **contar quantas vezes ela aparece**. Isso é a base para **agrupamentos e relatórios por categoria**, muito comuns em BI e ciência de dados.
Exemplo cotidiano: Compare com uma planilha em que o aluno quer saber quantos alunos há em cada turma — ele repetiria o processo de contagem manualmente, mas o laço faz isso automaticamente.

Mostre que o domínio de laços vem do **entendimento da lógica**, não da memorização.
Reforce que o analista deve sempre pensar:
Que dado estou percorrendo?
Qual condição preciso aplicar?
O que quero calcular no final?

Estimule que os alunos verbalizem o raciocínio antes de codificar.
Mostre que o mesmo pensamento se aplica a dados em planilhas, bancos SQL ou DataFrames — só muda a ferramenta.

Finalize reforçando que os laços de repetição são o **núcleo da automação analítica**.
Eles substituem tarefas repetitivas por um processo controlado e preciso — a essência da atuação em ciência de dados.

Slide 11

Orientações: o objetivo da seção **Colocando em prática** é levar os estudantes a aplicar os conhecimentos construídos durante a aula, incentivando-os a promover autonomia, senso crítico e criatividade.

Tempo previsto: 17 minutos.

Gestão de sala de aula: explique que o foco dessa atividade é pensar antes de codificar. Reforce que, na área de dados, o domínio vem da compreensão da lógica, e não da memorização da sintaxe.

Condução da dinâmica:
Apresentação (3 min):
Projete o contexto da atividade.
Leia os itens e reforce que o produto final será **um texto curto explicando o raciocínio lógico**, e não o código.
Execução da atividade (10 min):
Oriente as duplas a escreverem em caderno ou folha o passo a passo lógico.
Circule pela sala e incentive a argumentação entre os pares.
Observe se os alunos estão organizando o pensamento em sequência lógica (entrada → processamento → saída).
Por exemplo:
1. Para somar o total de vendas do dia:
O computador deve começar com um valor inicial de 0.
Depois, ele deve olhar uma por uma todas as vendas registradas no dia.
Para cada venda, deve pegar o valor e somar ao total que já tem.
Quando terminar de olhar todas as vendas, o valor final será o total de vendas do dia.
2. Para contar quantas vendas foram realizadas em cada cidade:
O computador deve olhar uma por uma todas as vendas registradas.
Para cada venda, ele deve verificar em qual cidade ela foi feita.
Se já tiver contado vendas dessa cidade, soma mais 1 no total dela.
Se for a primeira vez que aparece essa cidade, começa a contar com 1.
No final, terá uma lista com o nome de cada cidade e a quantidade de vendas feitas nela.

Continua

Slide 11

Encerramento (4 min):
Selecione **2 ou 3 duplas** para apresentar brevemente seus códigos e resultados .

Expectativas de respostas: as respostas devem conter **um raciocínio lógico estruturado**, não código.
Espera-se que os alunos descrevam ideias como:
Precisamos percorrer todos os pedidos e somar o valor de cada um. Depois, contar quantas vezes cada cidade aparece.

A dupla deve demonstrar **entendimento do fluxo do laço** e **clareza sobre o objetivo da repetição**.
O aprendizado esperado é compreender **como transformar um problema real em uma sequência lógica de instruções** — a base da programação aplicada a dados.

Slide 12 e 13

Orientações: o objetivo da seção **Pause e responda** é mobilizar os estudantes em relação aos conhecimentos apresentados e construídos em aula. É registro.

Tempo previsto: 2 minutos.

Resposta certa: Comparar e calcular valores.
Feedback: O laço for possibilita percorrer registros mistos (como cidade, produto e valor) e aplicar regras e cálculos automáticos sobre cada item.

Slide 14

Orientações: a seção **O que nós aprendemos hoje?** tem o objetivo de reforçar e esclarecer os principais conceitos discutidos, incentivando os estudantes a recordarem e articularem os principais pontos da aula.

Tempo previsto: 2 minutos.

Gestão da sala de aula:
mantenha um ambiente de diálogo aberto e respeitoso;
seja direto e objetivo nas explicações para manter a atividade dentro do tempo estipulado;
engaje os estudantes rapidamente, pedindo confirmações ou reações breves às definições apresentadas;

Condução da dinâmica:
explique aos estudantes que esse é um momento de reflexão e esclarecimento sobre os conceitos abordados na aula;
informe que será feita uma rápida revisão para assegurar o entendimento correto das definições e dos conceitos;
apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando a explicação com as informações que achar necessárias;
finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.

O que é esperado: os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.

Slide 15

Orientações: a seção **Saiba mais** apresenta recursos extraclasse para que os estudantes complementem sua formação. Incentive-os a continuarem explorando o tema da aula.

Tempo previsto: 1 minuto.

Educação Profissional Paulista

Técnico em **Ciência de Dados**

GOVERNO DO ESTADO
DE SÃO PAULO

Educação Profissional Paulista

Técnico em **Ciência de Dados**

GOVERNO DO ESTADO
DE SÃO PAULO

Lógica de programação e fundamentos de python

Laços de repetição: for

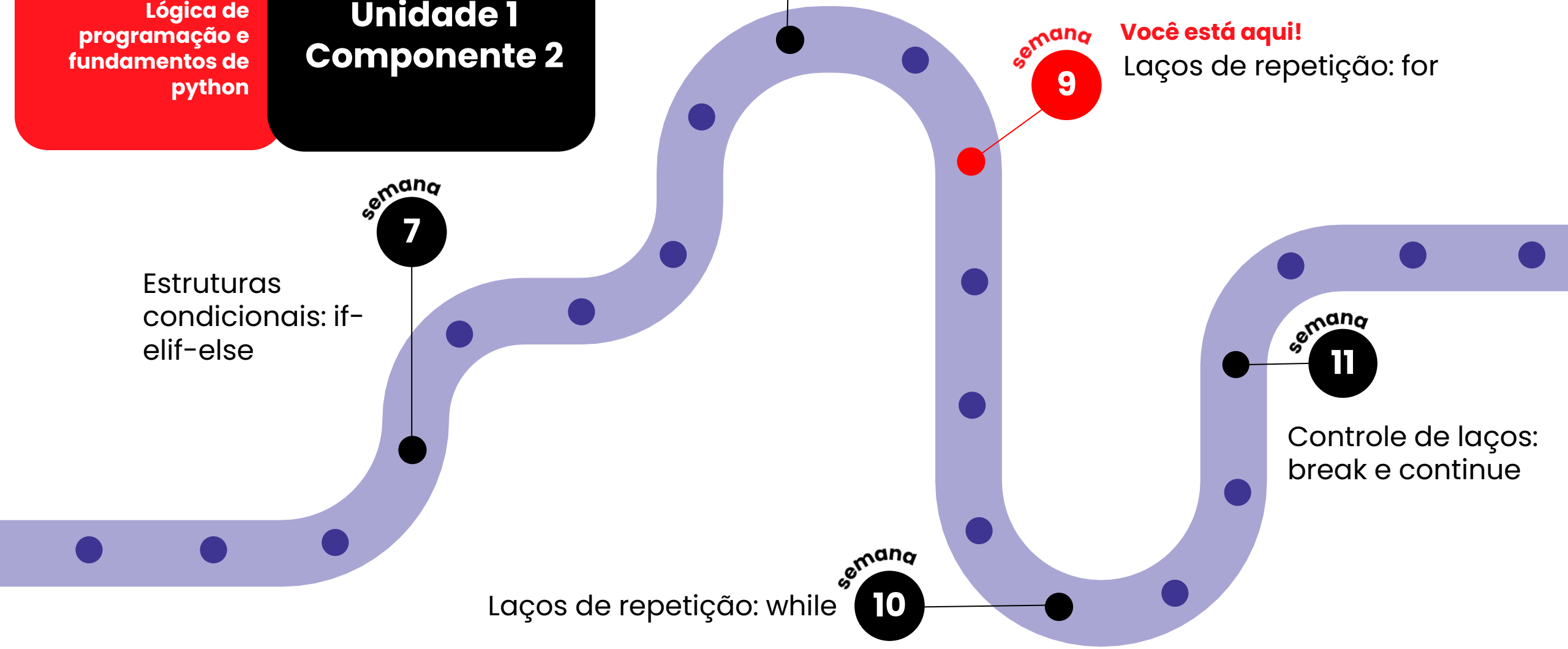
Aula 4: Análise de algoritmos com estruturas integradas

Código da aula: DADOSANO1C2B2S9A4

GOVERNO DO ESTADO DE SÃO PAULO

Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2



Lógica de programação e fundamentos de python

Mapa da Unidade 1 Componente 2

Você está aqui!

Laços de repetição: for

9

Aula 4: Análise de algoritmos com estruturas integradas

Código da aula: DADOSANO1C2B2S9A4

Objetivos da aula

- Integrar estruturas de repetição e decisão para resolver problemas mais complexos.

Recursos didáticos

- Roteiro de atividade prática.
- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e à internet.
- Ferramentas de codificação para python.

Duração da aula

50 minutos.

Habilidades técnicas

- Integrar laços de repetição e estruturas condicionais para resolver problemas complexos.

Habilidades socioemocionais

- Posicionar-se com equilíbrio e escutar diferentes propostas.

Diálogo

Relembre

Aplicações dos laços

- Por que é importante pensar no raciocínio lógico antes de escrever o código?
- Qual é a principal vantagem de usar laços em tarefas de análise de dados?
- O que o laço aninhado possibilita em uma análise de dados?

Colocando em **prática**

Registro

Controlando repetições com range()

Materiais necessários

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática;
- Software Jupyter Notebook ou Google Colab.

Passo a passo

1. Baixe o arquivo “DADOSANO1C2B2S9A4AP_ALUNO” que contém o roteiro de atividade prática.
2. Execute cada passo em conjunto com o professor.
3. Depois, resolva as atividades propostas.
4. Faça upload do arquivo .ipynb com suas respostas, no AVA.

Duplas

40 minutos

Registre seu trabalho no AVA.

Baixe o roteiro dessa atividade / Baixe o material de apoio dessa atividade

© Getty Images

O que nós **aprendemos hoje?**

Então ficamos assim...

1. Compreendemos que o comando range() permite controlar o número exato de repetições em um laço for.

2. Observamos como o range() pode ser usado para percorrer posições específicas em listas, simulando varreduras de dados.

3. Aplicamos essa lógica para automatizar verificações e cálculos parciais em conjuntos de informações.

Saiba mais

Para entender mais sobre a função range acesse o link disponível abaixo e assista o vídeo.

DEVTALKS. **Falando um pouco sobre a função Range em Python**. YouTube, 24 jul. 2023. Disponível em: https://www.youtube.com/watch?v=GDUZlhfUZgs. Acesso em: 5 nov. 2025.

Referências da aula

MENEZES, N. N. C. **Introdução à programação com Python: algoritmos e lógica de programação para iniciantes**. São Paulo: Novatec, 2019.

PYTHON SOFTWARE FOUNDATION. **Documentação Python 3.14.0**, [s.d.]. Disponível em: https://docs.python.org/pt-br/3/. Acesso em: 5 nov. 2025.

RAMALHO, L. **Python fluente: programação clara, concisa e eficiente**. São Paulo: Novatec, 2023.

Identidade visual: imagens © Getty Images

Orientações ao Professor

Slide 05

Orientações: professor, indicamos a seguir o tempo previsto para o desenvolvimento das seções desta aula.
Relembre: 10 minutos.
Colocando em prática: 37 minutos.
O que nós aprendemos hoje: 2 minutos.
Saiba mais: 1 minuto.

Slide 06

Orientações: a seção **Relembre** tem o objetivo de retomar conhecimentos já adquiridos por meio de atividades simples/exercícios que abordem conceitos previamente trabalhados com os estudantes.

Tempo previsto: 10 minutos.

Gestão de sala de aula: assegure-se de que todos os estudantes tenham a oportunidade de participar. Se necessário, faça rodízio ou direcione perguntas a estudantes que estejam menos ativos para garantir a participação de todos. Mantenha um ambiente de respeito, onde todas as opiniões são valorizadas, garantindo que todos se sintam confortáveis para expressar seus pontos de vista. Conclua a atividade resumindo as principais ideias discutidas e vinculando-as aos objetivos de aprendizagem da aula.

Condução da dinâmica: faça as perguntas do relembre para os alunos e os incentive a participar e relembrar os conceitos apresentados nas aulas anteriores. É importante reforçar bem os conceitos aprendidos, pois os alunos vão precisar estar com eles bem fixados para as demais aulas.

Expectativa de respostas:
Resposta 1: Porque o código é apenas a forma de traduzir o pensamento em comandos. Sem compreender o raciocínio, estamos apenas repetindo estruturas sem entender como o programa resolve o problema.

Resposta 2: Os laços permitem automatizar ações repetitivas, como somar valores ou contar registros, tornando o processo mais rápido e confiável.

Resposta 3: Ele permite cruzar informações entre duas listas, como cidade e produto, criando combinações e contagens úteis para relatórios e agrupamentos.

Slide 07

Orientações: o objetivo da seção **Colocando em prática** é levar os estudantes a aplicar os conhecimentos construídos durante a aula, incentivando-os a promover autonomia, senso crítico, criatividade.

Tempo previsto: 40 minutos.

Gestão de sala de aula: retome brevemente o conceito de range() e explique que ele é usado para controlar o início, o fim e o número de repetições em um laço for.

Condução da dinâmica:
Apresentação (8 min):
Mostre rapidamente no projetor o Exemplo 1 do notebook (verificação de registros), destacando o uso de range(len(lista)) e como ele percorre posições específicas da lista.
Explique que nesta atividade as duplas irão usar o range() para percorrer e somar valores em contextos simples de análise de dados.
Execução da atividade (25 min):
Oriente-as a:

1. Ler e discutir o enunciado juntos antes de começar o código.
2. Identificar o que deve ser controlado (quantas repetições, quais posições da lista).
3. Escrever e testar o código passo a passo, verificando se o resultado corresponde ao esperado.

Circule pela sala, verificando se os alunos:
Entenderam como o range() limita o laço;
Conseguem relacionar os índices às posições reais dos dados;
Usam nomes de variáveis claros (total_manha, total_tarde), reforçando boas práticas.
Reforce que os alunos devem realizar a entrega no AVA.
Encerramento (7 min):
Selecione **2 ou 3 duplas** para apresentar brevemente seus códigos e resultados.

Expectativas de respostas:
Estão descritas no **roteiro do professor - DADOSANO1C2B2S9A4AP_PROFESSOR**

Slide 08

Orientações: a seção **O que nós aprendemos hoje?** tem o objetivo de reforçar e esclarecer os principais conceitos discutidos, incentivando os estudantes a recordarem e articularem os principais pontos da aula.

Tempo previsto: 2 minutos.

Gestão da sala de aula:
mantenha um ambiente de diálogo aberto e respeitoso;
seja direto e objetivo nas explicações para manter a atividade dentro do tempo estipulado;
engaje os estudantes rapidamente, pedindo confirmações ou reações breves às definições apresentadas.

Condução da dinâmica:
explique aos estudantes que esse é um momento de reflexão e esclarecimento sobre os conceitos abordados na aula;
informe que será feita uma rápida revisão para assegurar o entendimento correto das definições e dos conceitos;
apresente o slide com a definição sintética de cada conceito principal discutido na aula, ampliando a explicação com as informações que achar necessárias;
finalize resumindo os pontos principais e reiterando a importância de cada conceito e como ele se encaixa no contexto maior da aula.

O que é esperado: os estudantes devem sair da aula com um entendimento claro e preciso dos conceitos principais.

Slide 09

Orientações: a seção **Saiba mais** apresenta recursos extraclasse para que os estudantes complementem sua formação. Incentive-os a continuarem explorando o tema da aula.

Tempo previsto: 1 minuto.

Educação Profissional Paulista

Técnico em **Ciência de Dados**

Secretaria da Educação

SÃO PAULO
GOVERNO DO ESTADO

{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">TÉCNICO EM CIÊNCIA DE DADOS</h1>\n",
    "<h1 style=\"text-align: center;\">Roteiro de Atividade Prática</h1>\n",
    "
\n",
    "
\n",
    "\n",
    "**Componente -** Programação Aplicada à Ciência de Dados\n",
    "
\n",
    "**Unidade Curricular -** Lógica de Programação e Fundamentos de Python\n",
    "
\n",
    "**Tema da Semana -** Laços de repetição: for\n",
    "
\n",
    "**Semana 9**\n",
    "
\n",
    "**Aula 4 -** Análise de algoritmos com estruturas integradas"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "<h1 style=\"text-align: center;\">Controlando repetições com range()</h1> "
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Objetivos da atividade\n",
    "\n",
    "* Compreender como o comando range() controla a quantidade de repetições em um laço for.\n",
    "\n",
    "* Aplicar range() em situações de dados que exigem varreduras controladas, como percorrer índices de listas e repetir cálculos um número definido de vezes.\n",
    "\n",
    "* Desenvolver o raciocínio lógico para automatizar tarefas em análises de dados, evitando repetições manuais e erros operacionais."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [

```
    "### Base teórica\n",
    "\n",
    "#### Controlando laços com range()\n",
    "\n",
    "Em tarefas de dados, muitas vezes precisamos repetir uma ação um
número conhecido de vezes ou percorrer posições (índices) de uma
lista/tabela.\n",
    "\n",
    "O comando range() é a forma mais simples de controlar quantas
repetições o laço for executa.\n",
    "\n",
    "#### O que é range()\n",
    "\n",
    "range() cria uma sequência de números usada pelo for para contar
repetições.\n",
    "\n",
    "Ele não gera uma lista na tela; é um "contador" eficiente que o for
usa internamente.\n",
    "\n",
    "#### Formas de uso:\n",
    "\n",
    "* range(stop) → começa em 0 e vai até stop (exclusivo).\n",
    "\n",
    "* range(start, stop) → começa em start e vai até stop
(exclusivo).\n",
    "\n",
    "* range(start, stop, step) → idem, avançando de step em step (pode
ser negativo).\n",
    "\n",
    "> **Regra importante:** o stop é exclusivo (não é incluído). Por
exemplo, se o stop no 5 (range(5)) o último número será o 4, o número 5 é
excluído. Dessa forma, o range sempre conta do início até `fim-1`.\n",
    "\n",
    "#### Por que isso importa em dados?\n",
    "\n",
    "* Repetir N vezes uma ação (ex.: simular 10 leituras).\n",
    "\n",
    "* Percorrer índices de uma lista para acessar posição e valor (ex.:
validar campos, construir mensagens do tipo "linha X, coluna Y").\n",
    "\n",
    "* Criar varreduras estruturadas (ex.: "para cada linha", "para cada
coluna"), aproximando do raciocínio de tabelas.\n",
    "\n",
    "#### Padrões mentais úteis\n",
    "\n",
    "* **Contadores e acumuladores:** iniciar em 0 e ir somando ou
contando dentro do laço.\n",
    "\n",
    "* **Validação com decisão:** combinar for + range() com if para
checar condições a cada passo (ex.: "se valor for negativo, marcar como
inválido").\n",
    "\n",
```

"* **Índice + valor:** quando precisa do índice, use range(len(lista)); quando não precisa do índice, prefira percorrer os valores diretamente (mais legível).\n",
"\n",
"#### Boas práticas\n",
"\n",
"* Evite “números mágicos”: declare uma variável para o limite (ex.: total_linhas = 5) e use range(total_linhas).\n",
"\n",
"* Nomeie claramente contadores e acumuladores (cont_validos, soma_valores).\n",
"\n",
"* Passos coerentes: use step com critério (ex.: de 2 em 2 para amostragens, ou negativo para contagens regressivas).\n",
"\n",
"* **Leia antes de calcular:** em dados, primeiro percorra/inspecione, depois aplique regras (soma, contagem, filtro).\n",
"\n",
"> Com range(), você **decide o roteiro da repetição:** quantas vezes, de onde começa, onde termina e de quanto em quanto.\n",
">\n",
"> Na prática, isso permite **organizar varreduras controladas** sobre listas e estruturas de dados — base para a atividade de hoje."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"#### **Exemplo: verificando registros de uma lista de vendas**\n",
"\n",
"Imagine que você recebeu uma lista com os valores das vendas de um dia.\n",
"\n",
"Antes de gerar o relatório, você quer verificar cada valor, identificando o número do registro (posição) e o valor correspondente.\n",
"\n",
"O range() ajuda a controlar a contagem e acessar cada item corretamente."
]
},
{
"cell_type": "code",
"execution_count": 2,
"metadata": {},
"outputs": [
{
"name": "stdout",
"output_type": "stream",
"text": [
"Verificação dos registros de venda:\n",
"\n",
"Registro 1: R$250\n",

"Registro 2: R$320\n",
"Registro 3: R$180\n",
"Registro 4: R$400\n",
"Registro 5: R$150\n"
]
}
],
"source": [
"# Lista de vendas do dia (em reais)\n",
"vendas = [250, 320, 180, 400, 150]\n",
"\n",
"print(\"Verificação dos registros de venda:\\n\")\n",
"\n",
"# Usando range() para percorrer os índices da lista\n",
"for i in range(len(vendas)):   # range(5) → 0 a 4\n",
"   print(f\"Registro {i+1}: R${vendas[i]}\")\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"> **_Raciocínio por trás do código_**\n",
"> \n",
"> * len(vendas) retorna o tamanho da lista (5 itens).\n",
">\n",
"> * range(len(vendas)) cria uma sequência de 0 a 4, controlando a posição de leitura.\n",
">\n",
"> * i+1 é usado apenas para exibir a contagem a partir de 1, como em um relatório real.\n",
">\n",
"> * Essa é uma forma simples de validar registros e evitar erros em planilhas ou listas extensas."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"#### **Exemplo: somando valores automaticamente com controle de repetição**\n",
"\n",
"Agora, imagine que você precisa somar apenas os primeiros 3 registros da lista, simulando um teste de cálculo parcial.\n",
"\n",
"Com range(), você decide exatamente quantas repetições serão executadas."
]
},
{
"cell_type": "code",
"execution_count": 3,
"metadata": {},

"outputs": [
 {
  "name": "stdout",
  "output_type": "stream",
  "text": [
   "Soma parcial (3 primeiros registros): R$ 750\n"
  ]
 }
],
"source": [
 "# Lista de vendas\n",
 "vendas = [250, 320, 180, 400, 150]\n",
 "\n",
 "# Somar apenas os três primeiros registros\n",
 "total = 0\n",
 "\n",
 "for i in range(3):   # controla quantas vezes repetir\n",
 "   total += vendas[i]\n",
 "\n",
 "print(\"Soma parcial (3 primeiros registros): R$\", total)"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
 "> **_Raciocínio por trás do código:_**\n",
 "> \n",
 "> * range(3) limita o laço às 3 primeiras posições (0, 1, 2).\n",
 ">\n",
 "> * A cada repetição, o programa acessa a posição e soma o valor correspondente.\n",
 ">\n",
 "> * Essa técnica é útil quando é preciso testar partes de uma amostra antes de processar o conjunto completo.\n",
 ">\n",
 "> * O controle de repetições é essencial em tarefas de verificação, amostragem e validação de dados."
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
 "---\n",
 "### **Colocando em Prática**"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
 "### **Exercício 1 - Conferindo amostras de registros**\n",
 "\n",

"A equipe de dados da loja Tech+ recebeu um arquivo com valores de pedidos realizados ao longo do dia.\n",
"\n",
"Antes de calcular o total geral, você precisa verificar os 5 primeiros registros para garantir que os dados foram importados corretamente.\n",
"\n",
"**O que fazer:**\n",
"\n",
"1. Use um laço for com range() para mostrar os valores e o número do registro (ex.: “Registro 1: R$250”).\n",
"\n",
"2. Exiba apenas os 5 primeiros registros da lista abaixo.\n",
"\n",
"pedidos = [250, 180, 320, 400, 150, 280, 310, 200]\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"metadata": {},
"outputs": [],
"source": [
"# Escreva sua solução aqui\n",
"\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"---\n",
"#### **Exercício 2 - Calculando totais parciais**\n",
"\n",
"A mesma equipe precisa testar o cálculo parcial de vendas de cada turno (manhã e tarde).\n",
"\n",
"Você recebeu a lista de valores e deve gerar duas somas parciais, uma para os 4 primeiros registros (turno da manhã) e outra para os demais (turno da tarde).\n",
"\n",
"O que fazer:\n",
"\n",
"1. Use dois laços for com range(): um para somar os 4 primeiros registros e outro para os restantes.\n",
"\n",
"2. Mostre os totais de cada turno no final.\n",
"\n",
"vendas = [320, 250, 180, 400, 150, 200, 270, 350]\n",
"\n"
]
},
{
"cell_type": "code",

```
   "execution_count": 4,
   "metadata": {},
   "outputs": [],
   "source": [
    "# Escreva sua solução aqui\n",
    "\n"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "---\n",
    "#### <h1 style=\"text-align: center;\">Gabarito comentado</h1> "
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "#### Exercício 1 - Conferindo amostras de registros"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 4,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Conferência dos 5 primeiros registros:\n",
      "\n",
      "Registro 1: R$250\n",
      "Registro 2: R$180\n",
      "Registro 3: R$320\n",
      "Registro 4: R$400\n",
      "Registro 5: R$150\n"
     ]
    }
   ],
   "source": [
    "# Lista de pedidos recebidos\n",
    "pedidos = [250, 180, 320, 400, 150, 280, 310, 200]\n",
    "\n",
    "print(\"Conferência dos 5 primeiros registros:\\n\")\n",
    "\n",
    "# Laço controlado por range()\n",
    "for i in range(5):                      # repete 5 vezes (índices 0
a 4)\n",
    "    print(f\"Registro {i+1}: R${pedidos[i]}\")\n"
   ]
  },
```

```
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Explicação para o professor:_**\n",
    "> \n",
    "> * range(5) limita a repetição aos 5 primeiros índices (0 a 4).\n",
    ">\n",
    "> * i+1 é usado apenas para exibir o número do registro de forma
natural (começando em 1).\n",
    ">\n",
    "> * A cada repetição, o código acessa e mostra o valor do pedido
correspondente na lista.\n",
    ">\n",
    "> * Essa abordagem é útil para conferir amostras iniciais de dados,
evitando processar listas inteiras antes da verificação."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "#### Exercício 2 — Calculando totais parciais"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 5,
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Total da manhã: R$ 1150\n",
      "Total da tarde: R$ 970\n"
     ]
    }
   ],
   "source": [
    "# Lista de valores de vendas\n",
    "vendas = [320, 250, 180, 400, 150, 200, 270, 350]\n",
    "\n",
    "# Total da manhã (primeiros 4 registros)\n",
    "total_manha = 0\n",
    "for i in range(4):                      # índices 0, 1, 2, 3\n",
    "    total_manha += vendas[i]\n",
    "\n",
    "# Total da tarde (restante dos registros)\n",
    "total_tarde = 0\n",
    "for i in range(4, len(vendas)):         # de 4 até o final\n",
    "    total_tarde += vendas[i]\n",
    "\n",
    "print(\"Total da manhã: R$\", total_manha)\n",
```

```
    "print(\"Total da tarde: R$\", total_tarde)"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "> **_Explicação para o professor:_**\n",
    "> \n",
    "> * O primeiro laço (range(4)) soma apenas os quatro primeiros
valores da lista.\n",
    ">\n",
    "> * O segundo laço (range(4, len(vendas))) percorre o restante,
somando os valores das posições 4 a 7.\n",
    ">\n",
    "> * Essa divisão simula análises por turno, período ou amostra,
comuns em relatórios de dados.\n",
    ">\n",
    "> * O len(vendas) garante que o laço funcione mesmo se a lista tiver
tamanho diferente — boa prática para quem trabalha com dados dinâmicos."
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "base",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.4"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 4
}
```